nature protocols

https://doi.org/10.1038/s41596-023-00931-7

Protocol

Check for updates

Reconstruction, simulation and analysis of enzyme-constrained metabolic models using GECKO Toolbox 3.0

Yu Chen[1,2,8], Johan Gustafsson[1,8], Albert Tafur Rangel[1,3,8], Mihail Anton[4], Iván Domenzain[1], Cheewin Kittikunapong[1], Feiran Li[1,5], Le Yuan[1], Jens Nielsen[1,6] & Eduard J. Kerkhoven[1,3,7]

Abstract

Genome-scale metabolic models (GEMs) are computational representations that enable mathematical exploration of metabolic behaviors within cellular and environmental constraints. Despite their wide usage in biotechnology, biomedicine and fundamental studies, there are many phenotypes that GEMs are unable to correctly predict. GECKO is a method to improve the predictive power of a GEM by incorporating enzymatic constraints using kinetic and omics data. GECKO has enabled reconstruction of enzyme-constrained metabolic models (ecModels) for diverse organisms, which show better predictive performance than conventional GEMs. In this protocol, we describe how to use the latest version GECKO 3.0; the procedure has five stages: (1) expansion from a starting metabolic model to an ecModel structure, (2) integration of enzyme turnover numbers into the ecModel structure, (3) model tuning, (4) integration of proteomics data into the ecModel and (5) simulation and analysis of ecModels. GECKO 3.0 incorporates deep learning-predicted enzyme kinetics, paving the way for improved metabolic models for virtually any organism and cell line in the absence of experimental data. The time of running the whole protocol is organism dependent, e.g., ~5 h for yeast.

Key points

- Genome-scale metabolic models have the potential to predict changes in phenotype resulting from different environmental conditions. Their predictive power can be improved by including more information about the enzyme kinetics.
- GECKO is a program that gives users an automated and manual mechanism to incorporate relevant data. The protocol also describes how to tune the program to compensate for incorrect or missing values.

Key references

Domenzain, I. et al. *Nat. Commun.* **13**, 3766 (2022): https://doi.org/10.1038/s41467-022-31421-1

Li, F. et al. *Nat. Catal.* **5**, 662–672 (2022): https://doi.org/10.1038/s41929-022-00798-z

Sánchez, B. J. et al. *Mol. Syst. Biol.* **13**, 935 (2017): https://doi.org/10.15252/msb.20167411

[1]Department of Life Sciences, Chalmers University of Technology, Gothenburg, Sweden. [2]Key Laboratory of Quantitative Synthetic Biology, Shenzhen Institute of Synthetic Biology, Shenzhen Institute of Advanced Technology, Chinese Academy of Sciences, Shenzhen, China. [3]Novo Nordisk Foundation Center for Biosustainability, Technology University of Denmark, Lyngby, Denmark. [4]Department of Life Sciences, National Bioinformatics Infrastructure Sweden, Science for Life Laboratory, Chalmers University of Technology, Gothenburg, Sweden. [5]Institute of Biopharmaceutical and Health Engineering, Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen, China. [6]BioInnovation Institute, Copenhagen, Denmark. [7]SciLifeLab, Chalmers University of Technology, SE-412 96, Gothenburg, Sweden. [8]These authors contributed equally: Yu Chen, Johan Gustafsson, Albert Tafur Rangel. e-mail: eduardk@chalmers.se

Introduction

The use of genome-scale models is a valuable approach to mathematically explore metabolism, and has accelerated biotechnology, biomedicine and fundamental biology studies[1–5]. A conventional genome-scale metabolic model (GEM) is reconstructed from all metabolic reactions of the organism of interest and converted into a computational format (Fig. 1a). The model is based on a system of equations with variables being reaction rates, represented in a stoichiometric matrix (*S*-matrix) that contains stoichiometric coefficients of metabolites in reactions (Fig. 1b). The system can be solved by constraint-based methods[6], which impose constraints on reactions rates and thereby search for solutions within an allowable solution space where all possible metabolic states reside. Therefore, integration of more constraints into the model may shrink the solution space and further improve the predictive power of the model.

While the most widely used constraints are based on condition-dependent measurements such as uptake rates of nutrients, the addition of additional innate constraints on metabolic reactions have recently gained a lot of attention. In particular, constraints on the catalytic capacity of enzymes have been used to reduce the solution space, and several methods that integrate such constraints into conventional metabolic models have emerged, e.g., metabolism and expression-model[7], resource balance analysis[8], expression and thermodynamics flux[9], pcModel[10], flux balance analysis with molecular crowding[11], MetabOlic Modeling with ENzyme kineTics (MOMENT)[12], GECKO[13], short (s)MOMENT[14] and enzymatic constrained metabolic network model (ECMpy)[15]. Among them, GECKO[13,16], a method that enhances a Genome-scale metabolic model with Enzymatic Constraints using Kinetic and Omics data, has been widely adopted to build enzyme-constrained metabolic models (ecModels) by expanding on conventional GEMs for various organisms[17,18]. Based on the two predecessors, this protocol presents the latest GECKO version 3.0 and describes how to use it for reconstruction, simulation and analysis of ecModels. Note that GECKO can in principle convert not only GEMs, which are at genome scale, but also smaller-size metabolic models to ecModels. In this protocol, we focus on GEMs.

Development of the protocol

GECKO enables direct conversion of a conventional GEM into an enzyme-constrained version. The conversion is based on the equation $v \leq k_{cat} \times e$, where v represents the rate of the enzymatic reaction, k_{cat} the turnover number of the enzyme that catalyzes the reaction and e the concentration of the enzyme. This means that the minimum usage of the enzyme of the reaction is the reaction rate v over k_{cat} and thus mass usage is the rate v times the molecular weight (MW) of the enzyme over k_{cat}. Accordingly, each enzyme is included as a pseudo-substrate in the reaction that it catalyzes (Fig. 1c), and the model captures both the turnover number and MW of the enzyme (k_{cat} and MW, Fig. 1d).

Enzyme usage reactions

In addition, enzyme usage reactions are added into the model. In the absence of any proteomic data, all protein masses are drawn from a total protein pool. If quantitative data are available for any of the enzymes, this knowledge can be used to explicitly constrain the reactions catalysed by these enzymes, and the measured enzyme mass is subsequentially subtracted from the protein pool (Fig. 1c,d).

In GECKO versions 1 and 2, k_{cat} and MW were included as separate coefficients in metabolic and enzyme usage reactions[13,16], while in version 3, they are combined to form the stoichiometric coefficient: $-MW/k_{cat}$. This mathematical conversion reduces the chance that the program will predict extremely low fluxes in enzyme usage reactions and thereby could avoid numerical issues in some linear programming solvers, where the flux values in certain circumstances could otherwise be lower than the numeric tolerance of the linear problem solver to separate different flux values. This form links to the concept of protein cost that quantitatively reflects the protein mass required per flux of an enzymatic reaction[19]. Compared with conventional GEMs, the *S*-matrix is expanded and the model can predict enzyme usage besides the reaction rates (Fig. 1d).

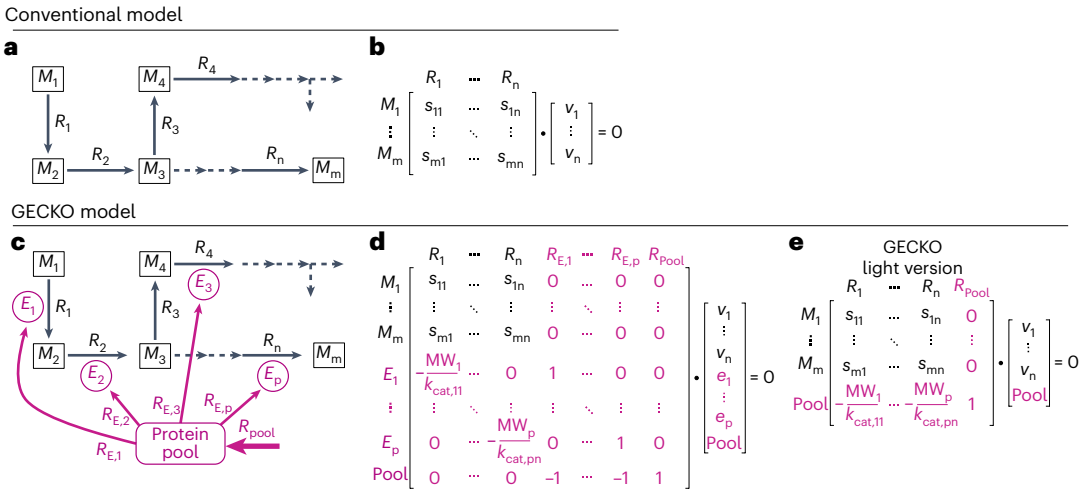

Fig. 1 | GECKO framework. **a**, A metabolic network of a conventional metabolic model, where *M* represents metabolites and *R* reactions. **b**, A system of equations derived from the metabolic network. In the *S*-matrix (stoichiometry matrix), rows represent metabolites and columns reactions. The variables (v) represent reaction rates. The vector on the right-hand side indicates concentration changes over time of a vector of involved metabolites, which are all assumed to be zero under steady-state conditions. **c**, The GECKO model includes enzymes (denoted as *E*), which draw protein mass from a shared protein pool via enzyme usage reactions. **d**, A system of equations in GECKO framework. The *S*-matrix is expanded, where enzymes and enzyme usage reactions are added, and the coefficients include enzyme turnover number (k_{cat}) and MW. The vector of variables is expanded to include enzyme usage levels and total protein pool. **e**, A system of equations in the GECKO light version, in which there is no enzyme and enzyme usage reaction but a pseudo-metabolite is included in the metabolic reaction to account for protein cost.

GECKO light

In addition, GECKO 3.0 offers an option to reconstruct a light ecModel[20], with similar formalism as other frameworks[14,21], i.e., the protein cost of the enzyme is included in the metabolic reaction while enzyme usage reactions are excluded (Fig. 1e). GECKO light has a much shorter calculation time than the full version and is therefore more practical for large-scale model reconstruction and analysis, e.g., for cell lines. While light ecModels are incapable of proteomics integration and enzyme usage prediction, because they do not include enzyme usage reactions, they give comparable results to full ecModels when only using a total proteome constraint.

DLKcat to provide predicted k_{cat} values

Given the essential role of k_{cat} values in ecModels, their availability and accuracy dominate the model performance. While k_{cat} values can be gathered from public databases such as the BRaunschweig ENzyme DAtabase (BRENDA)[22], the databases are far from complete, meaning that not every enzymatic reaction in an ecModel can be assigned with a measured k_{cat}. The coverage of measured k_{cat} values is small in model organisms and even worse in lesser studied organisms[23], which can explain why ecModels have only been reconstructed for relatively few organisms[18]. To address this, the deep learning-based model DLKcat has been demonstrated to be effective in predicting k_{cat} values for given enzyme–substrate pairs[24]. It was previously shown how the use of predicted k_{cat} values drastically improved the reconstruction of ecModels for 343 yeast/fungi species[24]. Therefore, GECKO can run DLKcat to provide predicted k_{cat} values as an alternative source. With this step, one can theoretically assign k_{cat} values to all enzymatic reactions and thus makes it possible to turn any conventional GEM into an ecModel.

Applications of GECKO

GECKO is mainly used to convert a conventional GEM to an ecModel by including enzyme constraints, which can considerably reduce the solution space and thereby the flux variability ranges in simulations[16,25,26]. Notably, this reduction of the solution space can effectively lead to accurate predictions of phenotypes such as overflow metabolism[13] and batch growth of microbes[13,27], which cannot be captured by conventional GEMs unless specific enzymatic,

transport or exchange reactions are constrained, such as oxygen uptake. Additionally, the reconstructed ecModels, attributed to the involved enzyme information, enable calculation and comparison of parameters, e.g., pathway protein costs[28] and flux control coefficients[29]. Lastly, the ecModels can be used to guide and interpret metabolic engineering strategies for overproduction of metabolites[27,30–33], and quantitatively characterize phenotype responses of microbes under industrially relevant conditions[13,16].

In addition, any constraint-based method[34] that fits a conventional GEM can also be applied to an ecModel as GECKO merely expands the *S*-matrix with additional pseudo-metabolites and pseudo-reactions while retaining linearity. The constraint-based methods do not only refer to typical ones, e.g., flux balance analysis (FBA)[35] and flux variability analysis (FVA)[36], but also those applied in strain design and optimization such as flux scanning based on enforced objective flux[37] as applied to guide heme overproduction[30]. Moreover, it has been demonstrated that GECKO fits with task-driven integrative network inference for tissues (tINIT)[38], one of the transcriptomics-based constraint methods that enable context-specific model reconstruction[39]. This has been shown by a sequential use of tINIT and GECKO with the GEM of *Homo sapiens*[40].

Comparison of GECKO 3.0 with other methods

It should be noted that the enzyme constraints can be incorporated by alternative methods such as sMOMENT[14] and ECMpy[15]. The applications of the resulting ecModels are similar to the ecModels by GECKO, e.g., improved predictions of phenotypes[14,41–43]. sMOMENT adds a pseudo-metabolite, i.e., protein pool, into metabolic reactions, and an exchange reaction for this protein pool, and the resulting model is therefore similar to the light ecModel by GECKO. The difference is that sMOMENT is based on Python and unable to construct the full ecModel.

Another Python-based tool, ECMpy, avoids adding enzymes into reactions but constrains the protein pool outside the *S*-matrix, therefore the resulting model has a smaller size compared with the full and light ecModel by GECKO. A limitation of defining the protein pool outside the *S*-matrix is that the enzyme constraints cannot be directly considered in constraint-based simulations that were developed for conventional GEMs.

GECKO differs from the alternative methods in the capability of proteomics data prediction and integration, and potential exploration of between metabolism and other processes, e.g., signaling[44]. In addition, the underlying model structures of ecModels as defined by GECKO are similar to conventional GEMs, which makes it possible to apply virtually any type of analysis or simulation that has previously been developed for conventional GEMs. Furthermore, by incorporating deep learning-predicted enzyme kinetics, GECKO 3.0 is capable of considerably improving the parameter coverage, which is currently the major obstacle using other methods.

Limitations

While following the same overall structure, ecModels are much larger than the starting conventional GEMs, which means that the computational cost in large-scale simulations is substantially larger. GECKO 3.0 addresses this limitation by providing light ecModels, where consideration of individual enzyme levels is disabled. There is, therefore, a trade-off between integration of proteomics data, or prediction of individual enzyme levels, and computational speed that needs to be considered.

The reduction in solution space by enzyme constraints integration improves predictions but may also lead to larger deviations from experimental observations due to uncertainties in the applied constraints. One uncertainty comes from the k_{cat} values, which are obtained from either reported measurements or DLKcat predictions. Measurements and predictions might sometimes be inaccurate, particularly for enzyme complexes[45], and in addition, almost all the sources reflect in vitro enzyme kinetics, which might differ from in vivo conditions. GECKO provides multiple strategies to test the resulting ecModel with known phenotype data, and suggests what constraints should be modified. Therefore, experimental data are necessary for high-quality reconstruction of ecModels.

Despite the inclusion of enzyme constraints, ecModels are still not able to predict all possible phenotypes due to the lack of other constraints. For example, metabolic fluxes can be influenced by allosteric regulation and posttranslational modifications, and integration

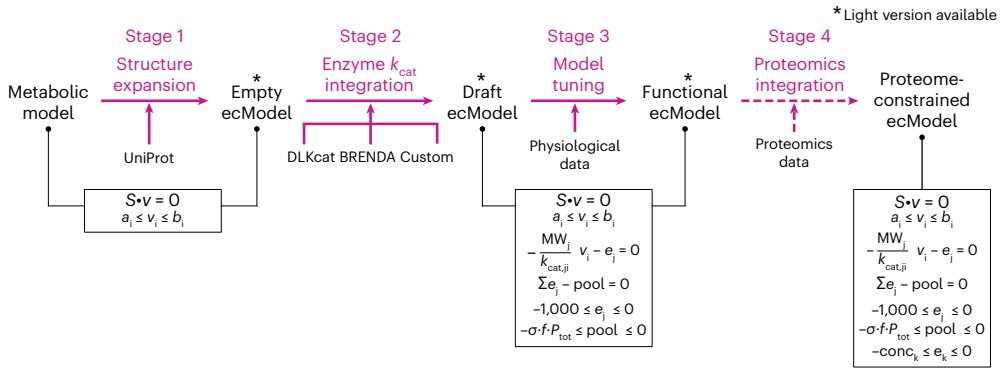

Fig. 2 | Workflow for ecModel reconstruction using GECKO 3.0. There are four stages associated with ecModel reconstruction. In Stage 1, the structure of the starting metabolic model is expanded without including any new constraints, and the resulting model is called empty ecModel. In Stage 2, enzyme information is integrated into the model, and the resulting model is called draft ecModel. In Stage 3, a functional ecModel is obtained by tuning the draft ecModel based on physiological data. In Stage 4, proteomics data, if available, can be integrated into the model, and the resulting model is called proteome-constrained ecModel.

of these biological processes can be expected to further reduce the solution space and hence increase prediction accuracy.

Overview of the procedure

In addition to a preparation stage, the procedure consists of five stages, where the first four are associated with ecModel reconstruction (Fig. 2) and the last one is simulation and analysis of ecModels.

The starting material for Stage 1 is a conventional GEM that will be converted to the ecModel format. At the end of Stage 1, most of the variable fields are empty and will be populated in Stage 2.

Before Stage 2 the enzyme data need to be assembled and formatted. It is very important that the enzyme information, including MWs, protein sequences and EC numbers, is well organized. These data can normally be collated from organism-specific entries from the UniProt[46] database. The Kyoto Encyclopedia of Genes and Genomes (KEGG)[47] database could also be an alternative source for gathering EC numbers.

In Stage 2, enzyme k_{cat} values are integrated into the empty ecModel, resulting in a draft ecModel, which thus introduces enzymatic constraints as can be seen in the mathematical formulation of the model. In Stage 3, model parameters are tuned based on physiological data if available, resulting in a functional ecModel. If there is proteomics data available, Stage 4 enables proteomics integration, resulting in a proteome-constrained ecModel, which thus has more constraints.

All of these stages are also applicable to the reconstruction of light ecModels, except for Stage 4 as proteomics integration does not fit light ecModels.

Stage 0: Steps 1–7, preparation of project file structure and data

This stage is to generate files and prepare data and parameters for ecModel reconstruction, simulation and analysis. The ideal starting material is a highly curated conventional constraint-based metabolic model of the organism of interest. If such a model is not available, one can reconstruct it from scratch or from template models following published protocols[48,49] within public toolboxes[34,50,51]. In addition to the file of the starting GEM, each ecModel requires a model adapter, which is a text file that is loaded as a MATLAB structure. The organism-specific data and parameters can be collated and populated into the adapter file.

Stage 1: Steps 8–14, expansion from a starting metabolic model to an ecModel structure

The starting GEM is expanded so that enzyme constraints can be implemented in later stages.

1. Reversible reactions are split into forward and backward reactions as enzyme kinetics are direction dependent.

2. Reactions that are catalyzed by isozymes are split into multiple reactions by individual isozymes due to differences in enzyme kinetics and MWs of isozymes (the light ecModel skips this and instead uses the lowest protein cost among isozymes.)
3. The protein pool is added as a pseudo-metabolite and an exchange reaction is added for it.
4. Enzymes are added as pseudo-metabolites, which draw protein resources from the protein pool by enzyme usage reactions (this step is not performed in the light version, because the protein pool is directly added into enzymatic reactions.)
   In this stage, an empty ecModel is obtained, in which ECs are not yet incorporated.

Stage 2: Steps 15–32, integration of k_{cat} into the ecModel structure

This stage is to integrate enzyme k_{cat} values together with the enzyme information that has been prepared into the empty ecModel. It is important to bear in mind that not every enzymatic reaction has a reported measurement. GECKO can directly retrieve reported data from the BRENDA database for the reaction if both the EC number and the substrate of the reaction of the organism are matched with the k_{cat} annotation. If there are multiple k_{cat} values matched for the reaction, then the maximum value is selected.

If the perfect match of all organism, substrate and EC number is not found, GECKO allows for k_{cat} matches by querying the BRENDA database based on a set of hierarchical matching criteria[16] as summarized here:

- If the organism is not matched, then the k_{cat} value of the phylogenetically closest organism is selected
- If the substrate is not matched, then the k_{cat} value for any alternative substrate is selected
- If the EC number is not found, then a fuzzy EC number, i.e., by introducing wildcards into the EC number, is used for matching
- Where no k_{cat} value is available in the BRENDA database, specific activities that are typically given in μmol per min per mg protein can be converted into k_{cat} values, as an alternative

In addition, the DLKcat package, which is independent of EC number, can complement the BRENDA-based hierarchical matches, particularly for less studied organisms, or even be used as a sole source of k_{cat} values, enabling considerable improvement of the k_{cat} coverage of ecModels[24].

While it is possible to use either of these two automated approaches to populate k_{cat} for all reactions, GECKO also allows convenient manual curation of the k_{cat} values so that the user can also assign k_{cat} values for reactions of interest from other sources, such as in vivo estimations[8,52–56] and predictions by other approaches[57].

Once the k_{cat} values are obtained, enzymes with their protein costs in the form of MW over k_{cat} are added into all enzymatic reactions in the model. The product of Stage 2 is a draft ecModel. This model is not functional, because the protein pool is unlimited and the enzyme constraint is, therefore, not active.

Stage 3: Steps 33–52, model tuning

To make the ecModel functional, a maximum constraint should be imposed on the protein pool. According to the original publication[13], the value of the maximum constraint is calculated by $\sigma \times f \times P_{tot}$, where sigma σ represents the average in vivo saturation of all enzymes involved in the model, f the mass fraction occupied by metabolic enzymes in total proteome and P_{tot} the total protein mass in the unit of gram per gram cell dry weight. In cases where the value of one or more factors is unknown, a default value of 0.5 can be used for any of the factors. Consequentially, the default maximum constraint without knowledge of any factors is $0.5 \times 0.5 \times 0.5 = 0.125$. In addition, there will be some uncertainty regarding the k_{cat} values, which might lead to incorrect predictions. Therefore, in this stage the model can be tuned by entering already known physiological data.

In GECKO 3.0, the draft ecModel can be tuned by adjusting the protein pool and/or k_{cat} of specific enzymes. There are several ways in which this can be done:

The easiest approach is to increase the maximum constraint on the protein pool. If the protein pool is allowed to increase without constraint, the model will be able

to reach a higher growth rate. While this is an effective way of increasing the growth rate, it might result in an unrealistically high protein pool, mostly as a consequence of extremely low and incorrect k_{cat} values of some key enzymes (protein pool values higher than one signify that at least one of the factors σ, f and P_{tot} is greater than one, which is in theory impossible). Another option is to replace extremely low k_{cat} values with an arbitrary value (e.g., tenfold of the original value), which is also easy to implement and can be combined with the first option. In addition to these easier but less biologically meaningful options, GECKO[16] provides the functionality to identify the enzymes that most limit growth and substitute their k_{cat} with higher values. By tuning the model with the strategies above, a functional ecModel is obtained, which is constrained by a protein pool that can be distributed between individual enzyme usages.

It is important to bear in mind that model tuning requires physiological data, particularly growth rate data. If there is no such data, e.g., in the case of large-scale model reconstruction for various cell lines, only draft ecModels can be obtained. While the draft model may not be functional for simulating realistic metabolic fluxes due to the undefined upper bound on the protein pool, it can serve as a useful tool to compare cellular parameters, e.g., the comparison of protein costs[28]. Even if no experimental data are available, functional ecModels could be obtained by assuming an arbitrary value for the protein pool and increasing low k_{cat} values to ensure that reasonable simulations are achieved.

Stage 4: Steps 53–65, integration of proteomics data into the ecModel

Enzyme usage reactions allow for the integration of proteomics data into the ecModels reconstructed by GECKO, leading to proteome-constrained ecModels that are context specific. Due to, e.g., measurement errors in proteomics data and uncertainties in enzyme kinetics, the ecModels are most likely to be overconstrained by proteomics data, resulting in an infeasible solution. This can be addressed by using a proteomics integration algorithm that loosens the constraints on the enzymes with great control coefficients[16], where the control coefficient of an enzyme is calculated by the change in the growth rate over the change in the concentration of the enzyme.

Stage 5: Steps 66–79, simulation and analysis of ecModels

Due to the similar mathematical formulations between ecModels and conventional GEMs, the constraint-based methods that are widely used in conventional GEMs are compatible with ecModels, such as FBA[35] and FVA[36]. Notably, the estimated rates for enzyme usage reactions represent the minimum enzyme levels that are required for maintaining the fluxes through the corresponding metabolic reactions. If proteomics data are available, one could investigate whether certain enzymes are limiting by comparing the simulated and measured enzyme levels.

Materials

Equipment

- Personal computer with operating systems including MacOS, Windows and Unix
- Internet connection to access KEGG (https://www.genome.jp/kegg/), UniProt (https://www.uniprot.org/), PubChem (https://pubchem.ncbi.nlm.nih.gov/) and GitHub (https://github.com/SysBioChalmers/GECKO/)—the data obtained from these resources can also manually be downloaded and/or reconstructed on another computer, but direct download through GECKO functions is most convenient

Software

- MATLAB version 2019b or above (MathWorks, https://mathworks.com/products/matlab.html); no additional MathWorks toolboxes are required

- RAVEN toolbox (https://github.com/SysBioChalmers/RAVEN) version 2.8.3 or above—the RAVEN toolbox[51] provides built-in functions that are leveraged to reconstruct ecModels, and some RAVEN functions are also used to simulate and analyse ecModels
- Gurobi Optimizer (Gurobi Optimization LLC, https://www.gurobi.com/solutions/gurobi-optimizer/) is strongly recommended for simulations (free academic license available). Alternatively, SoPlex as part of the SCIP Optimization Suite (Zuse Institute Berlin, https://scipopt.org/) can be used.
- Docker (Docker Inc, https://www.docker.com/) for running DLKcat

Equipment setup

Conventional metabolic model

The conventional constraint-based metabolic model can either be at the genome scale (i.e. GEM), or describe a smaller metabolic network. The steps of the procedure assume that the starting model is a GEM. The model should be provided in the community standard Systems Biology Markup Language (SBML) format L3V1 FBCv2 (https://synonym.caltech.edu/documents/specifications/level-3/version-1/fbc /). Whether the model is in valid SBML format can be tested through SBML Validator (https://synonym.caltech.edu/validator_servlet/).

- A well-curated model will result in a well performing ecModel. However, this protocol will work on any model, no matter whether it is for a model organism
- Conventional constraint-based metabolic models can be obtained from literature or databases e.g., BioModels database (https://www.ebi.ac.uk/biomodels/)

RAVEN Toolbox

- Installation instructions are provided on the RAVEN Wiki: https://github.com/SysBioChalmers/RAVEN/wiki/Installation. Briefly, the RAVEN repository is either downloaded via git clone, as ZIP-archive from GitHub or installed as a MATLAB AddOn
- Instructions on how to install the recommended solver Gurobi are described on the RAVEN wiki: https://github.com/SysBioChalmers/RAVEN/wiki/Installation#solvers
- After finishing all installation instructions, run installation checks in MATLAB with:

```
checkInstallation;
```

GECKO Toolbox

- The latest GECKO release can be installed as MATLAB Add-On: https://mathworks.com/help/matlab/matlab_env/get-add-ons.html
- Alternatively, GECKO toolbox can be obtained via git clone of the GECKO repository:

```
git clone --depth=1 https://github.com/SysBioChalmers/GECKO
```

- As a third option, a ZIP-archive with GECKO toolbox can be downloaded from the GitHub repository (https://github.com/SysBioChalmers/GECKO/releases). The ZIP-archive should be extracted to a disk location where the user has read- and write-access rights
- After git clone or extracting the ZIP-archive, the user should navigate in MATLAB to the GECKO folder by modifying the following command to match the GECKO folder location

```
cd('path/to/GECKO')
```

- GECKO is installed with the command that adds GECKO (sub-)folders to the MATLAB path:

```
GECKOInstaller.install
```

- If desired, a removal command is available as:

```
GECKOInstaller.uninstall
```

Table 1 | Files required to get an ecModel

File stored in data folder	Description	Steps
ComplexPortal.json	Information relating to enzyme complexes retrieved from the Complex Portal, automatically downloaded by 'getComplexData'	12 and 13
customKcats.tsv	Manually curated data containing k_{cat} values for specific enzymes or reactions in the ecModel. Should be manually constructed by the user, based on other data sources. Example is given in 'full_ecModel' tutorial	28
DLKcat.tsv	Input file for DLKcat, which will filled in with predicted k_{cat} values. Generated by 'writeDLKcatInput'	23 and 24
fluxData.tsv	Experimental data, e.g., growth rate, protein content and carbon source uptake, used to constrain ecModel during proteomics integration. Metabolic exchange fluxes are given in mmol/gDCWh, uptake rates are negative values while excretion rates have positive values. Growth rate and carbon source uptake rate are essential. Is not required for the initial ecModel reconstruction (Stages 1–3)	59
kegg.tsv	KEGG information for organism defined in adapter file. Must contain UniProt protein ID, gene, KEGG gene ID, EC number, MW, pathway and sequence. Used for assigning EC numbers. Automatically downloaded from the KEGG website by 'loadDatabases'	5–7 and 17
paxDB.tsv	Protein abundance file used to compute the *f* factor, as a proxy to the mass fraction of proteins accounted for in an ecModel out of the total protein content in cells. Can be manually retrieved from https://pax-db.org. If the file is absent, a default *f* factor of 0.5 is used	32
proteomics.tsv	Measured protein levels from one or more experiments, in the unit of mg protein/gDCW. Used for proteomics integration. The first column contains UniProt protein identifiers, while each subsequent column contains protein levels from replicates from individual experiments. E.g., columns 2–3 contain protein levels in duplicate from growth under control conditions, while columns 4–6 contain proteins levels in triplicate from growth with some treatment imposed on the culture	54–56
smilesDB.tsv	SMILES for metabolites in ecModel.mets. Automatically generated when 'findMetSmiles' queries the PubChem database	21–23
uniprot.tsv	UniProt data for the organism defined in the model adapter. Must contain protein identifier, gene identifier, EC number, MW and sequence. Automatically downloaded from UniProt database by 'loadDatabases'	5–7, 11 and 17
uniprotConversion.tsv	Conversion table required if UniProt does not have a field that contains the genes in the same format as used in 'ecModel.genes'. First column is ecModel.genes, while second column is UniProt identifier. Should be manually constructed by the user, based on other data sources. An example is given in the 'light_ecModel' tutorial	6

Docker

Installation instructions are available at https://docs.docker.com/get-docker/.

Tutorial code

All code included in this protocol is also available in 'GECKO/tutorials'. Most code, related to a full ecModel is demonstrated in 'full_ecModel/protocol.m', which will reconstruct and analyze a tutorial ecModel for *Saccharomyces cerevisiae*. In addition, 'light_ecModel/protocol.m' contains example code for making a light ecModel of the generic human-GEM. Note that both tutorials generate ecModels that are only suitable for use as part of the respective tutorials. The code reconstructing each ecModel may contain functions that are applied for demonstration purposes and dependent on how the ecModel is intended to be used. The ecModel may require additional curation and evaluation.

Procedure

Stage 0: preparation of project file structure and data

● **TIMING 15 min**

Project file structure

1. Generate a new GECKO project file structure with:

```
startGECKOproject();
```

This automatically prepares a new folder in the location of the user's choice. This folder contains four subfolders: 'code', 'data', 'models' and 'output'. The folders 'code' and 'data' are used to keep custom codes and data for ecModel reconstruction and analysis, respectively. A description of the files in data folder is given in Table 1. The folders 'models' and 'output' save reconstructed ecModels and simulation results, respectively.

Table 2 | Parameters specified in model adapter

obj.params entries modelAdapter	Example from full_ecModel tutorial	Explanation
bioRxn	'r_4041'	Reaction identifier for biomass or growth pseudo-reaction, i.e., the reaction in the model that represents the polymerization of biomass macromolecules or precursors, and whose flux value can be interpreted as the growth rate in h^{-1}
c_source	'r_1714'	Reaction identifier for glucose exchange reaction (or other preferred carbon source). Shown here is glucose exchange
complex.taxonomicID	559292	Taxonomic identifier as available from Complex Portal (https://www.ebi.ac.uk/complexportal/home)
convGEM	fullfile(obj.params.path, 'models','yeast-GEM.xml');	Path to the starting conventional GEM file that this ecModel will be based on
enzyme_comp	'cytoplasm'	Compartment name in which the added enzymes should be located, should match 'ecModel.compNames'
f	0.5	Fraction of enzymes in the ecModel in comparison to all proteins in the cell (unit: g enzyme/g protein). 'calculateFfactor()' can calculate this for a proteomics dataset
kegg.geneID	'kegg'	The gene identifier or database link from KEGG that corresponds to the genes in the ecModels, e.g., for human-GEM this should be 'Ensembl'
kegg.ID	'sce'	Organism ID in KEGG database, selected at: https://www.genome.jp/kegg/catalog/org_list.html
org_name	'saccharomyces cerevisiae'	Scientific name of the organism, used in by 'fuzzyKcatMatching'
path	fullfile(findGECKOroot, 'tutorials', 'full_ecModel');	Path to the directory where all ecModel-specific content can be found. Set in Step 8
Ptot	0.5	Total protein content in the cell (unit: g protein/gDCW)
sigma	0.5	Average enzyme saturation factor σ
uniprot.geneIDfield	'gene_oln'	Field name on UniProt that contains the gene identifiers that are used in the ecModel. It should be one of the 'Returned Field' entries under 'Names & Taxonomy' at: https://www.uniprot.org/help/return_fields
uniprot.ID	'UP00000231'	Identifier of the dataset on UniProt (either by taxonomy (https://www.uniprot.org/taxonomy) or proteome (https://www.uniprot.org/proteomes)) that provide a complete coverage of genes in the ecModel
uniprot.type	'proteome'	Whether a taxonomy or proteome is provided in uniprot.ID
uniprot.reviewed	true	Whether only reviewed data from UniProt should be considered. Reviewed data has highest confidence in their annotation, but coverage might be (very) low for nonmodel organisms

2. Store the starting GEM. We recommend using the 'models' subfolder for convenience and reproducibility, but other paths could also be used. For the remainder of this protocol, this model-specific folder 'tutorials/full_ecModel', will be referred to as the adapter folder, as it is the location where the model adapter is found.

Model adapter

3. By running 'startGECKOproject()' in Step 1, a template model adapter is generated and located in the new adapter folder. Within MATLAB, the model adapter acts as a class that inherits the 'ModelAdapter' base class, and it is possible to override standard behaviors implemented in the base class to adapt to the specific model.
4. Before initiating ecModel reconstruction, populate the model adapter file with organism- and model-specific information. An overview is presented in Table 2. If a particular parameter cannot be defined, e.g., because the information is absent or not relevant, comment out the line defining this parameter using %, for instance if the organism of interest is not available on KEGG.

```
% obj.params.kegg.ID = '';
```

5. Query the UniProt and KEGG databases using appropriate model adapter parameters following the guidelines described in 5A and 5B. The UniProt database is the source of enzyme MWs and amino acid sequences (this information is essential), while the KEGG database can be used to assign EC number annotations (this information is useful but optional). To ensure that this information is accurate and collated correctly, it is critical to carefully select the parameters by

which UniProt and KEGG are queried. The most crucial is that the data gathered from UniProt and KEGG refer to genes with the same style identifiers as those used in the starting GEM. Some critical considerations for each data source are listed in A and B.

(A) **UniProt**

▲ **CRITICAL** It is possible to identify data by either proteome (i) or taxonomy (iv). The option proteome is preferred as it tends to avoid redundant matches between gene and protein identifiers.

(i) Search for the organism-specific proteome datasets on UniProt https://www.uniprot.org/proteomes.

(ii) If multiple proteome datasets are available for the same organism, preferentially choose a reference proteome (https://www.uniprot.org/proteomes?facets=proteome_type%3A1&query=%2A).

(iii) When a potential proteome is identified, examine the UniProt page for one of its proteins to check that it refers to its corresponding gene with the same style identifiers as used in the starting GEM. This can typically be found under 'Names & Taxonomy—Gene names'.
For example, searching for *Saccharomyces cerevisiae* yields 93 UniProt proteomes, of which one is the reference proteome for *S. cerevisiae* strain S288c (proteome UP00000231). On the UniProt page for one of its proteins (e.g., https://www.uniprot.org/uniprotkb/D6VTK4/entry#names_and_taxonomy) the gene identifier styled as YFL026W can be found, which matches the same style as used in the yeast-GEM starting GEM. This style of gene name is marked as Ordered locus name, which in the application programming interface (https://www.uniprot.org/help/return_fields) can be accessed via gene_oln. In the model adapter, 'uniprot.type' is set as 'proteome', 'uniprot.ID' as 'UP00000231' and 'uniprot.geneIDfield' as 'gene_oln'.

(iv) While for most organisms a proteome is defined (based on all proteins in their genome), taxonomy (https://www.uniprot.org/taxonomy/) can also be used to identify UniProt datasets. Here, the taxonomy identifier should be as specific as possible.
For example, for *S. cerevisiae* the taxonomy identifier 4932 (https://www.uniprot.org/taxonomy/4932) refers to 322 different strains with a total of 53,526 proteins. Instead, 559292 (https://www.uniprot.org/taxonomy/559292) refers to strain S288c with 6,735 proteins. In that case, uniport.type is 'taxonomy' and 'uniprot.ID' is '559292'.

(B) **KEGG**

(i) Choose the correct KEGG-specific three- or four-letter species identifiers (e.g., ID in the model adapter) from https://www.genome.jp/kegg/catalog/org_list.html. For *Homo sapiens* the KEGG identifier is 'has'.

(ii) To check that the correct gene identifier is linked, refer to the genome entry page (e.g., https://www.genome.jp/entry/hsa) that contains a link to the list of KEGG genes. For *H. sapiens*, the genes are sequentially numbered (e.g., https://www.genome.jp/entry/hsa:100008586), which does not match the gene identifiers used in the *H. sapiens* model human-GEM. Instead, under 'Other DBs' are Ensembl gene identifiers provided (e.g., ENSG00000236362). For human-GEM, 'kegg.geneID' in the model adapter should therefore be 'Ensembl' instead of 'KEGG'.

6. If an organism is included in UniProt but no satisfactory 'uniport.geneIDfield' can be identified in Step 5 (i.e., the gene identifiers in the model are not included in any of the UniProt fields), manually construct a conversion table. This table is located at 'data/uniprotConversion.tsv' and contains columns of model genes and UniProt identifiers. For example, the *Escherichia coli* model iML1515 (http://bigg.ucsd.edu/models/iML1515) uses gene identifiers styled as b0008, but these are individually mentioned as such in UniProt. However, KEGG seemingly has both of these identifiers for *E. coli* genes. Reconstruct the required 'uniprotConversion.tsv' with the following code, which requires that the model adapter is loaded (Step 8):

```
DB = loadDatabases('kegg');
fID = fopen(fullfile(params.path, 'data', 'uniprotConversion.tsv'), 'w');
```

```
output = transpose([DB.kegg.keggGene, DB.kegg.uniprot]);
fprintf(fID, '%s\t%s\n', 'genes', 'uniprot');
fprintf(fID, '%s\t%s\n', output{:});
fclose(fID);
```

While the example for *E. coli* above demonstrates how the conversion table can be gathered from KEGG, for other organisms this might require the use of different data sources. For human-GEM, which is also provided with GECKO as example at 'tutorials/light_ecModel', a conversion table is distributed together with the starting GEM, which is converted to the 'uniprotConversion.tsv' file as detailed in 'tutorials/light_ecModel/protocol.m'.

7. If an organism is not included in UniProt or KEGG, preventing the definition of appropriate UniProt and KEGG parameters in Step 5, manually reconstruct the database files. Assuming accessible information from other sources, use the 'data/kegg.tsv' and 'data/uniprot.tsv' files from 'full_ecModel' as templates on how to organize the data. Use artificial UniProt and KEGG IDs in these reconstituted database files, but the real MW and sequence of each protein are required to enable the remainder of the protocol to function.

Stage 1: expansion from a starting metabolic model to an ecModel structure

● **TIMING 15 min**

8. The model adapter with model-specific information is required for many GECKO functions. Instead of repeatedly specifying it as input parameter, set a default model adapter using the 'ModelAdapterManager'. This default model adapter is then used by all GECKO functions during that MATLAB instance, unless another model adapter is explicitly mentioned. Set the default model adapter by directing toward the model adapter file, which is here assumed to be located at 'GECKO/tutorials/full_ecModel/YeastGEMAdapter.m':

```
adapterLocation = fullfile(findGECKOroot, 'tutorials',
'full_ecModel', 'YeastGEMAdapter.m');
ModelAdapterManager.setDefault(adapterLocation);
```

If desired, explicitly use the model adapter as input parameter when running GECKO functions, for instance, if multiple ecModels are simulated in the same MATLAB instance. In that case, load the model adapter(s) with:

```
ModelAdapter = ModelAdapterManager.getDefault();
```

Give a quick reference to the parameters contained in the model adapter by:

```
params = ModelAdapter.getParameters();
```

For the remaining part of this protocol, it is assumed that a default model adapter is set.
▲ **CRITICAL STEP** If any changes are made to the model adapter after setting it as default in MATLAB—e.g., a change is made in the file 'YeastGEMAdapter.m'—the model adapter should again be loaded as default adapter. Run the 'ModelAdapterManager.setDefault' function again as described in the step:

```
ModelAdapterManager.setDefault(adapterLocation);
```

9. To initiate reconstruction of a new ecModel, load the conventional GEM into MATLAB. If the location of the model is specified in the model adapter ('obj.params.convGEM'), then load the model with:

```
model = loadConventionalGEM();
```

Both YAML and XML files are supported. To load a model stored at a different location or without using model adapter, use the usual RAVEN command for XML files:

```
model = importModel('path/to/modelFile.xml');
```

Or YAML files:

```
model = readYAMLmodel('path/to/modelFile.yml');
```

▲ **CRITICAL STEP** By default, GECKO loads models through RAVEN. If the model is manually loaded in MATLAB through the COBRA toolbox[34], recognizable by the 'model.rules' field, convert it to the required RAVEN format, recognizable by the 'model.metComps' field, with 'ravenCobraWrapper()'.

10. Decide whether to reconstruct a full or light ecModel. This decision affects the general structure of the model, and the two versions are not interconvertible. While light ecModels are smaller and thereby faster in simulations, a full ecModel is required to enable the constraining of individual enzymes by their concentration from e.g., proteomics data.
11. Run one of the two following commands:
full ecModel:

```
[ecModel, noUniprot] = makeEcModel(model);
```

light ecModel:

```
[ecModel, noUniprot] = makeEcModel(model, true);
```

The program will then run. The 12-step algorithm is summarized in Box 1. The result of this step is an empty ecModel: the model structure (Table 3) is changed to allow consideration of enzyme constraints, but no such constraints are applied yet. While running 'makeEcModel', a warning might be reported that refers to how gene associations are specified in the starting GEM. This warning can be ignored if you are confident that the gene associations are correct. The warnings do not prevent the creation of an ecModel.
▲ **CRITICAL STEP** Occasionally, not all model genes can be found in the UniProt database. In such cases, the noUniprot cell array contains those model genes without match in the UniProt database. This will be reported by the pipeline as a Warning. If noUniprot contains a large number of genes—e.g., more than ten—it could be worth to consider if a different UniProt taxonomy or proteome identifier (Step 5) is more suitable to gather the required data for the model.
12. (Optional) While in Step 11, 'ecModel.ec.rxnEnzMat' is populated with single subunits for enzyme complexes, in reality the copy number of each subunit in an enzyme complex can vary. GECKO supports adding such information from the Complex Portal (https://www.ebi.ac.uk/complexportal/complex/organisms) when available using the taxonomic identifier specified in the model adapter as 'obj.params.complex.taxonomicID', while Complex Portal only has a limited number of allowed taxonomic identifiers. Download and apply Complex Portal data with:

```
complexInfo = getComplexData();
[ecModel, foundComplex, proposedComplex] =
applyComplexData(ecModel, complexInfo);
```

Note that 'getComplexData' does not require any input parameters, and similar behavior is also observed for multiple other GECKO functions. Here, the required parameters are gathered from the model adapter, which earlier was set to a default model adapter, so that it does not have to be specified as input parameter.

BOX 1

Extension of conventional GEM to empty ecModel

To enable the constraining of metabolic reactions with enzyme constraints, the conventional GEM is converted into an empty ecModel through up to 12 steps, some of which are skipped for light models. These steps are listed here to indicate how the structure of the ecModel is different from the starting conventional GEM.

1. Any potential gene associations are removed from pseudo-reactions: as these are not assumed to be enzymes that catalyze realistic metabolic reactions, they should not be considered by enzyme constraints. Such reactions are identified by either reaction name containing 'pseudoreaction', or reaction identifier being specified in 'data/pseudoRxns.tsv'.
2. Any irreversible reactions that are only allowed to carry a negative flux (lower bound < 0 and upper bound = 0) are inverted.
3. An 'ecModel.rev' vector, which indicates whether reactions are reversible or not, is defined from the vectors of lower and upper bounds.
4. Nearly all reversible reactions are split into forward and reverse reactions, to yield an irreversible ecModel. The new (previously reverse) reactions will have _REV appended to its reaction identifier, while the reaction identifier of the forward reaction will remain unchanged. The only exception are exchange reactions, which completely retain their original form and are not split into two reactions, therefore still being allowed to carry a negative flux.
5. (Skipped for light ecModels) Reactions catalyzed by isozymes (indicated by or in the gene associations in 'ecModel.grRules') are split, so that each reaction is catalyzed by one enzyme or enzyme complex. The new reaction will have their reaction identifier suffixed with _EXP_ followed by a sequential number. Example: reaction r_0001 catalyzed by two isozymes will have two reactions, r_0001_EXP_1 and r_0001_EXP_2. If a backward reaction r_0001_REV exists, this will give two more reactions: r_0001_REV_EXP_1 and r_0001_REV_EXP_2. As this step is skipped for light ecModels, their reaction identifiers remain unchanged from how they were defined in the starting GEM.
6. An empty 'ecModel.ec' structure is constructed, which will eventually be populated with information about, e.g., enzymes, k_{cat} values and associate enzymes to the correct reactions (Table 3).
7. Enzyme details such as MW and amino acid sequence are added to the 'ecModel.ec' structure. This information is gathered from UniProt, which is automatically obtained using the UniProt related parameters in the model adapter.
8. The 'ecModel.ec.rxnEnzMat' indicate which reactions are catalyzed by which enzymes. Momentarily, this is indicated by a 1, while at a later stage this can be modified to reflect potential different numbers of subunits that make an enzyme complex.
9. (Skipped for light ecModels) The enzymes are added as pseudo-metabolites to the ecModel, they will now appear also in 'ecModel.mets' with the identifier prefix prot_ followed by the protein identifier (as specified in 'ecModel.ec.enzymes'). For instance *S. cerevisiae* enolase gene YHR174W with (UniProt) protein identifier P00925 appears as pseudo-metabolite prot_P00925.
10. As previously described in the summary of Stage 1, a protein pool pseudo-metabolite is added to 'ecModel.mets'.
11. (Skipped for light ecModels) Add usage reactions for the enzyme pseudo-metabolites, which replenish enzyme pseudo-metabolites that are used by catalyzing reactions. The identifiers of the usage reactions are usage_ prefix followed by the metabolite identifier of the enzyme e.g., usage_prot_P00925.
12. An exchange reaction for the prot_pool pseudo-metabolite is added.

▲ **CRITICAL STEP** If Step 12 is omitted, or if information is missing for a particular enzyme complex, the ecModel will assume that each subunit has a stoichiometry of one, which is however not an unrealistic assumption for most complexes.

13. Carefully inspect the content of 'proposedComplex' and consider whether manual curation of the respective ecModel gene association might be appropriate. This may require additional literature study.

    ▲ **CRITICAL STEP** The model derived from 'applyComplexData' only integrates complex data for a reaction if a full match (100%) is found between its ecModel gene association and the data retrieved from Complex Portal. If no full match is found, 'proposedComplex' will suggest complexes that have a partial match. A complex from Complex Portal can be suggested by either of two criteria: (i) at least 75% of the ecModel genes of a reaction match with a complex in Complex Portal or (ii) a complex in Complex Portal contains more subunits than the genes that are associated to an ecModel reaction.
14. Save the ecModel to disk; this can be done at any point in the procedure. To retain all ecModel content, including the 'ecModel.ec fields', store the ecModel in YAML format. 'saveEcModel' automatically does this in the adapter folder, while the more generic 'writeYAMLmodel' can also store the file in any desired folder. To load the model back into MATLAB, use 'loadEcModel' or the generic 'readYAMLmodel'.

Table 3 | Internal structure of enzyme-related information in the ecModel

ecModel.ec field	Data type	Size	Description
rxns	string array	m	Reaction identifiers gathered directly from ecModel (after expansion and making irreversible)
rxnEnzMat	Matrix	$m \times n$	Somewhat comparable to rxnGeneMat, but Enz refers to 'ecModel.ec.enzymes' entries. Positive integers in this matrix indicate the number of enzyme (subunits) that are annotated to each of the reactions
kcat	Float vector	m	Gathered from various sources, see below. One value for each reaction-enzyme (complex) combination, sec^{-1}
source	String array	m	Where k_{cat} value is derived from, e.g.,'dlkcat', or 'brenda', 'standard' or 'custom'
notes	String array	m	Any notes the user wants to add
eccodes	String array	m	EC numbers, gathered directly from ecModel and/or via e.g., UniProt/KEGG. Only used for fuzzy k_{cat} GECKO matching
genes	String array	n	Gene identifiers, corresponding to 'ecModel.genes', matching the columns in rxnEnzMat
enzymes	String array	n	UniProt protein identifiers for the enzymes that are derived from the matching entries in genes
mw	String array	n	MW for each enzyme, in Dalton
sequence	String array	n	Amino acid sequence for each enzyme
concs	Float vector	n	Measured concentration of each enzyme in mg/gDCW

The structure is similar for full and light ecModels but differ in how the reaction-related fields are populated. In the full ecModels, the whole model is expanded, where each isozyme gets a separate reaction, and each such reaction is mapped to a reaction in the structure. In light ecModels, the reactions are not split up per isozyme, but the reactions in the structure are. There are thus several reaction entities in the structure for some reactions in the starting GEM structure.

```
saveEcModel(ecModel, 'ecModel.yml');
writeYAMLmodel(ecModel, 'C:\path\to\ecModel.yml');
ecModel = loadEcModel('ecModel.yml');
ecModel = readYAMLmodel('C:\path\to\ecModel.yml');
```

To enable constraint-based analysis in other software packages, it might be more suitable to exchange the model in SBML format (with XML file extension). For this, use 'exportModel' and 'importModel', but be aware that this file does not retain the 'ecModel.ec' fields.

```
exportModel(ecModel, 'C:\path\to\ecModelFull.xml');
ecModel = importModel('C:\path\to\ecModelFull.xml');
```

▲ **CRITICAL STEP** Only the YAML format contains the 'ecModel.ec' fields that are required to make modifications on the ECs of the ecModel.

Stage 2: integration of k_{cat} into the ecModel structure

● **TIMING 1 h**

15. Decide which source(s) of k_{cat} data you are going to use. GECKO can consider the four sources of k_{cat} data listed below. Any combination of source(s) can be used in any order, but at least one of the k_{cat} gathering approach should be conducted.
    - Through fuzzy matching with the BRENDA database (https://www.brenda-enzymes.org/)
    - By deep learning prediction with DLKcat
    - From a list of manually curated custom k_{cat} values
    - Assignment of a standard k_{cat} value

Fuzzy matching with BRENDA database

16. Gather reaction-specific k_{cat} data from the BRENDA database, where the reactions are matched by EC numbers and substrate names. As this approach is intended for flexible matching with either wildcards in the EC number or k_{cat} from other substrates, it is also known as fuzzy matching.
17. Assign EC numbers to reactions. This is done either by reading them from the 'model.eccodes' field in the conventional GEM, if such a field exists, or otherwise by gathering them from annotations in the UniProt and KEGG databases.
    - If the starting GEM has manually curated EC numbers, populate 'ecModel.ec.eccodes' with model-derived EC numbers using 'getECfromGEM'. For reactions without

EC number annotation, query UniProt/KEGG with 'getECfromDatabase' to get the highest coverage. As 'getECfromDatabase' will overwrite any entries in 'ecModel.ec.eccodes', it is necessary to first define which entries were not populated with EC numbers after running 'getECfromGEM':

```
ecModel = getECfromGEM(ecModel);
noEC = cellfun(@isempty, ecModel.ec.eccodes);
ecModel = getECfromDatabase(ecModel, noEC);
```

- If the starting GEM is not annotated with a 'model.eccodes' field; it does not contain standard EC numbers (i.e., four groups of digits separated by a period); or the user has low confidence in the accuracy of the 'model.eccodes' field as it is not a well-curated model, only run 'getECfromDatabase':

```
ecModel = getECfromDatabase(ecModel);
```

- If desired, the EC numbers derived from the database can be transferred to the model.eccodes fields with 'copyECtoGEM'.

18. After populating 'ecModel.ec.eccodes' with corresponding EC numbers, gather k_{cat} from BRENDA:

```
kcatList_fuzzy = fuzzyKcatMatching(ecModel);
```

In this process, BRENDA is queried for k_{cat} values by EC number, substrate and organism.

19. Look at the results. The output from this function (and later functions) is a 'kcatList' structure. This object documents which potential k_{cat} could be assigned to the various reactions, together with information on how precise the match is for each k_{cat} (i.e., number of required EC number wildcards, if matched by substrate and/or organism). It is common, even for model organisms such as *S. cerevisiae*, that no full match can be found in the BRENDA database. In those cases, k_{cat} values from partial organism and/or substrate matches are returned. If no k_{cat} is found for a particular EC number, wildcards in the EC number are introduced: e.g., EC 2.4.2.3 (uridine phosphorylase) becomes 2.4.2.- (pentosyltransferase).

Deep learning prediction with DLKcat

20. Gather k_{cat} through deep learning prediction with DLKcat, based on enzyme sequence information and substrate structural information in simplified molecular-input line-entry system (SMILES) format.
21. As starting GEMs rarely come with SMILES annotation, gather these by querying PubChem with 'findMetSmiles', thus creating 'data/smilesDB.tsv', or manually populating the file 'data/smilesDB.tsv' in the adapter folder.

```
[ecModel, noSMILES] = findMetSmiles(ecModel);
```

▲ **CRITICAL STEP** The 'findMetSmiles' function reports which percentage of unique metabolites could be assigned with SMILES annotation. If this percentage is below 100%, as is almost always the case, it is worthwhile to inspect the 'noSMILES' list of metabolite names. For example, in some models the metabolite name is suffixed by the metabolite formula (even while SBML has a dedicated field for this information), which precludes matching with PubChem. Manual curation of the 'ecModel.metNames' field is required to resolve such issues.

22. Define what metabolites should be used by DLKcat. The DLKcat prediction does not run natively in MATLAB but requires Python. However, GECKO can write the required input file for DLKcat. As part of this function, in accordance to how DLKcat functions[24], currency metabolites that occur in pairs e.g., adenosine triphosphate (ATP) versus adenosine diphosphate (ADP) and NADH versus NAD, and a selection of small molecules, e.g., Fe^{2+} are

excluded when writing the DLKcat input file. Nonexhaustive lists of such metabolites and molecules are provided under 'GECKO/databases' as 'DLKcatCurrencyMets.tsv' and 'DLKcatIgnore.tsv', while one can provide ecModel-specific metabolite names by having files with those same names under data in their adapter folder. Note that currency metabolites are only excluded if the reactions have other reactants (after small-molecule removal), such as the case for ATP synthase.

23. Write the DLKcat input file to 'data/DLKcat.tsv' under the adapter folder with 'writeDLKcatInput', which contains, e.g., amino acid sequences for proteins and SMILES for metabolites.

```
writeDLKcatInput(ecModel);
```

▲ **CRITICAL STEP** Any 'DLKcat.tsv' file generated will be specific for either the full or the light version of the ecModel, as the file contains the reaction identifiers from 'ecModel.ec.rxns' that will be required when loading the predicted k_{cat} values back into MATLAB (see below). Therefore, separate 'DLKcat.tsv' files need to be constructed for either of the two ecModel versions.

24. Run the DLKcat prediction pipeline with 'runDLKcat', which automatically download and start a Docker image:

```
runDLKcat();
```

Note that no other input parameters are given, as the function assumes that the DLKcat input file is written to 'data/DLKcat.tsv' in the adapter folder.

25. To construct a 'kcatList' structure, load back the outcome from DLKcat into MATLAB:

```
kcatList_DLKcat = readDLKcatOutput(ecModel);
```

Merge DLKcat and BRENDA structures

26. If k_{cat} values are gathered from both DLKcat (Step 25) and BRENDA database (through fuzzy matching, Step 18), merge the two 'kcatList' structures to increase the k_{cat} enzyme coverage.

```
kcatList_merged = mergeDLKcatAndFuzzyKcats(kcatList_DLKcat, kcatList_
fuzzy);
```

During this process, single k_{cat} values are assigned to each reaction, where priority is given to those k_{cat} that were obtained from BRENDA by full EC number match. By default, mismatches on organism and/or substrate are allowed, but this can be modified by providing additional input parameters. If no exact EC number match is available, the k_{cat} derived from DLKcat is used. If DLKcat was not able to predict a k_{cat} value for a reaction (for example, if its substrate had no SMILES assigned), then EC number wildcard matches from fuzzy matching are allowed, if available. For instance, instead of assigning a k_{cat} to pyruvate oxidase by querying EC 1.2.3.3, it will query EC 1.2.3.- that includes any oxidoreductase acting on the aldehyde or oxo group of donors with oxygen as acceptor, irrespective of reactant. From this group, the highest k_{cat} value will be selected. A more detailed description of the fuzzy matching algorithm is included in GECKO 2 (ref. 16).

27. To integrate k_{cat} data into the ecModel, run 'selectKcatValue' with a 'kcatList' as input, which could be derived from any source in previous steps. Here, the merged 'kcatList' populates 'ecModel.ec.kcat' with the relevant k_{cat} values:

```
ecModel = selectKcatValue(ecModel, kcatList_merged);
```

Provide custom k_{cat} values

28. Custom k_{cat} values can be provided as alternative or sole source of k_{cat} values. These k_{cat} values can be based on manual curation, potentially based on results from later in this procedure, or obtained through another analysis or prediction procedure, such as

in vivo apparent enzyme turnover numbers. Document the custom k_{cat} values in 'data/customKcats.tsv' in the adapter folder that are mapped to reactions and/or enzymes, and add directly to 'ecModel.ec.kcat' with:

```
[ecModel, rxnUpdated, notMatch] = applyCustomKcats(ecModel);
```

This function overwrites any existing values in 'ecModel.ec.kcat' for matching reactions, assuming that the custom k_{cat} values are of higher confidence. Pay attention if 'notMatch' contains reactions, as their k_{cat} values had not been successfully applied to the ecModel. A reason for this can be that the gene association between the model and the input file match <50%. To resolve such cases, inspect and manually curate the input file.
▲ **CRITICAL STEP** Functions that populate 'ecModel.ec.kcat', for instance 'selectKcatValue' and 'applyCustomKcats', but also other functions that are introduced below, can be run in any order. Unless otherwise specified, they will overwrite existing values in 'ecModel.ec.kcat', and document in 'ecModel.ec.source' where the current k_{cat} was derived, e.g., from 'brenda' or 'custom'.

Mean k_{cat} of isozymes

29. (Optional) Situations may occur where k_{cat} values of a particular reaction are not defined for all isozymatic variants. The reaction variant that is associated to the isozyme without k_{cat} will then have no protein cost. Consequentially, this reaction variant will be preferred over the other reaction variants that have a protein cost. To avoid this, k_{cat} values across isozymes are reasonable estimates for isozymes with no other k_{cat} knowledge available. Such missing k_{cat} values in 'ecModel.ec.kcats' can be substituted by the mean k_{cat} of their isozymes. To do so, run:

```
ecModel = getKcatAcrossIsozymes(ecModel);
```

Note that this situation is not relevant for light ecModels, where only the most efficient isozyme is considered.

Assign a standard k_{cat} value

30. (Optional) Reactions that do not have genes associated in the starting GEM (empty entries in 'model.grRules') cannot have enzyme constraints added by GECKO, due to the lack of enzyme data (i.e., MW and k_{cat} value). For those reactions (with the exception of exchange, spontaneous, transport and pseudo-reactions), a standard k_{cat} value and a standard pseudo-enzyme (prot_standard) can be included to constrain their catalytic capacity. Various standard k_{cat} values are calculated based on (i) the mean of k_{cat} values of the reactions involved in the subsystem in which a particular reaction belongs to or (ii) the mean of all k_{cat} values in 'ecModel.ec.kcat' if no subsystem is defined for that reaction. To follow the GECKO formulation that uses MW/k_{cat}, a standard MW is also calculated as median of all the proteins in the organism, and a protein usage reaction 'usage_prot_standard' is added to the ecModel. To do so, run:

```
[ecModel, rxnsMissingGPR, standardMW, standardKcat]
= getStandardKcat(ecModel);
```

The standard MW and subsystem agnostic standard k_{cat} are reported in 'standardMW' and 'standardKcat', respectively.

Apply the ECs to the model

31. After populating 'ecModel.ec.kcats' with various sources of k_{cat} values, apply the ECs with:

```
ecModel = applyKcatConstraints(ecModel);
```

During this step the 'ecModel.S' matrix is modified to directly include the protein cost (Fig. 1), based on the k_{cat} from 'ecModel.ec.kcats', the MWs from 'ecModel.ec.mw'

and, if relevant, multiplied by the enzyme complex stoichiometry as obtained from 'applyComplexData'. The output 'ecModel' has the enzymes directly involved as pseudo-substrates, to reflect enzyme usage. This function can be run at any point to (re-) apply the k_{cat}, MW and complex data as ECs.

Constrain the protein pool exchange reaction

32. The enzyme pseudo-metabolites that are included as cosubstrates in the above function are, by default, replenished from the total protein pool. The protein pool exchange reaction should therefore be constrained by a realistic value. This constraint is calculated from the σ, P_{tot} and f parameters in the model adapter, which stand for the average enzyme saturation, total protein content and fraction of proteins that are included in the model, respectively. To set the protein pool constraint, run:

```
ecModel = setProtPoolSize(ecModel);
```

If quantitative proteomics data are available, for instance via PaxDB, store the data in 'data/paxDB.tsv' and calculate the f factor with

```
f = calculateFfactor(ecModel);
```

Then use this anew f factor to set the protein pool constraint with:

```
ecModel = setProtPoolSize(ecModel, [], f);
```

Stage 3: model tuning

● **TIMING 15 min**

Simulate growth rate

33. After applying ECs, it is necessary to examine whether the ecModel can simulate a realistic growth rate. In particular, microorganisms should be able to reach their maximum growth rate if nutrient supply is not limiting. This can be simulated in the ecModel by removing constraints on nutrient exchange and maximizing biomass production, which can be done by functions from RAVEN toolbox (for a longer list of relevant RAVEN functions, see Box 2). To allow for unconstrained uptake of carbon source, run:

```
ecModel = setParam(ecModel, 'lb', params.c_source, -1000);
```

Or by directly referring to the glucose exchange reaction in the *S. cerevisiae* ecModel:

```
ecModel = setParam(ecModel, 'lb', 'r_1714', -1000);
```

34. In this example, the glucose exchange reaction is defined as 'glucose[e] <=>'. A positive, forwards flux through the reaction thereby represents a dissipation from the system. Consequentially, a negative flux through this reaction signifies glucose exchange into the system, which will allow for uptake into the cytoplasm. Modify the constraint on glucose uptake by setting the lower bound of the exchange reaction, which is a negative number to allow for negative fluxes. Query the direction of a reaction with:

```
constructEquations(ecModel, 'r_1714');
```

35. Set growth as the objective function, predict the flux distribution and print the growth rate:

```
ecModel = setParam(ecModel, 'obj', params.bioRxn, 1);
sol = solveLP(ecModel);
bioRxnIdx = getIndexes(ecModel, params.bioRxn, 'rxns');
fprintf('Growth rate: %f /hour\n', sol.x(bioRxnIdx))
```

BOX 2

Selection of RAVEN toolbox functions

For use on both conventional GEMs and ecModels.

- Set the upper bound of a reaction with reaction identifier 'r_0003' (see 'ecModel.rxns') to the value ten:

```
ecModel = setParam(ecModel, 'ub', 'r_0003', 10);
```

- Set the lower bound of a reaction with reaction identifier 'r_0003' (see 'ecModel.rxns') to the value zero:

```
ecModel = setParam(ecModel, 'lb', 'r_0003', 0);
```

- Set the objective function to maximize the flux through the biomass reaction, whose reaction identifier is indicated in 'params.bioRxn' (see 'ecModel.rxns'):

```
ecModel = setParam(ecModel, 'obj', params.bioRxn, 1);
```

- Perform FBA, where the objective function is optimized:

```
sol = solveLP(ecModel);
```

- Perform parsimonious FBA[63], where the objective function is optimized and the total sum of flux is minimized:

```
sol = solveLP(ecModel, 1);
```

- Inspect the (nonzero) fluxes through exchange reactions, based on results obtained from FBA (note that this includes usage reactions for enzymes that are constrained by their concentration, but not enzymes that draw from the protein pool, as protein pool usage is the exchange reaction in the latter case):

```
printFluxes(ecModel, sol.x)
```

- Inspect the (nonzero) fluxes through all reactions, based on results obtained from FBA:

```
printFluxes(ecModel, sol.x, false)
```

- Export the results to a Microsoft Excel file named 'filename.xlsx' with:

```
exportToExcelFormat(ecModel, 'filename.xlsx');
```

  While this file will not contain any content that is stored in the 'ecModel.ec' structure, the file is convenient to, e.g., quickly find reaction identifiers.
- Export the ecModel to an SBML file named 'filename.xml' for use in other constraint-based model software packages:

```
exportModel(ecModel, 'filename.xml');
```

36. Examine the predicted growth rate. In this example, the ecModel reaches a growth rate that is far lower than 0.41 h^{-1} (which is the maximum growth rate of *S. cerevisiae*, that is entered in the model adapter as 'obj.params.gR_exp'). This indicates that the model needs to be modified, i.e. tuned.
37. Confirm the reason why the ecModel did not reach the experimental maximum growth rate. Three reasons can be systematically checked:
    - The stoichiometry of the metabolic network is not fully correct
    - The protein pool exchange is too tightly constrained
    - Incorrect (too low) k_{cat} values are assigned to reactions

Network stoichiometry limits growth rate

38. Investigate the stoichiometry of the metabolic network itself by predicting the growth rate in the conventional GEM (i.e., without the enzyme constraints) with the same exchange flux constraints. To do so, run:

```
model = loadConventionalGEM();
model = setParam(model, 'lb', params.c_source, -1000);
model = setParam(model, 'obj', params.bioRxn, 1);
sol = solveLP(model);
bioRxnIdx = getIndexes(model, params.bioRxn, 'rxns');
fprintf('Growth rate: %f /hour\n', sol.x(bioRxnIdx))
```

    If the conventional GEM is also not able to reach the maximum growth rate, then the starting GEM contains errors in its stoichiometry or constraints that should be resolved before using it to reconstruct an ecModel.

Too tight protein pool constraint

39. Confirm that the protein pool constraint set in Step 32 is realistic. Inspect the σ, P_{tot} and f parameters in the model adapter and modify them to more realistic values if discrepancies are observed (Step 4).

40. Regardless of whether the protein pool parameters are realistic (Step 39), the simplest solution to enable the ecModel to reach the intended growth is to relax the constraint on the protein pool usage reaction, thereby relieving all enzyme contstraints at once. To release the protein pool constraint, set the lower bound to −1,000 (for reaction direction, see Step 34):

```
ecModel = setParam(ecModel, 'lb', 'prot_pool_exchange', -1000);
```

41. As the ecModel has now neither a protein pool constraint nor nutrient constraint (Step 33), it would be reasonable to predict the lowest protein pool usage that still supports the experimental maximum growth rate. To do this, run:

```
ecModel = setParam(ecModel, 'lb', 'r_4041', 0.41);
ecModel = setParam(ecModel, 'obj', 'prot_pool_exchange', 1);
sol = solveLP(ecModel);
protPoolIdx = strcmp(ecModel.rxns, 'prot_pool_exchange');
fprintf('Protein pool usage is: %.0f mg/gDCW\n', abs(sol.x(protPoolIdx)))
```

▲ **CRITICAL STEP** Due to the direction of the exchange reaction (Step 34), minimization of protein pool usage is implied by using '1' (and not '−1') as objective coefficient.

42. Set the predicted protein pool usage as constraint on the protein pool exchange reaction, before reverting the previously asserted objective function and constraint of the growth rate:

```
ecModel = setParam(ecModel, 'lb', protPoolIdx, sol.x(protPoolIdx));
ecModel = setParam(ecModel, 'lb', 'r_4041', 0);
ecModel = setParam(ecModel, 'obj', 'r_4041', 1);
```

While this approach always results in an ecModel that reaches the intended growth rate (assuming that the conventional GEM is also able to simulate this), it often leads to the protein pool usage reaction set to unrealistic protein content as mentioned before. It is therefore recommended to instead increase k_{cat} values of some reactions.

Sensitivity tuning of k_{cat} values

43. In contrast to Step 42, it is advised to rather iteratively increase k_{cat} values of enzymes that limit growth, until the desired growth rate is reached. At this stage, the enzyme constraints are dominated by the overarching protein pool constraint. Therefore, in each deterministic iteration, the most-limiting enzyme is identified as the one demanding the largest fraction of the protein pool. Consequentially, in each iteration, one k_{cat} is increased, and while the selected k_{cat} is not per se the most deviating from its in situ value, it is the k_{cat} whose value contributes most to overconstraining the model. First revert the protein pool constraint as set in Step 42 to a more realistic value (Step 32), tune the k_{cat} values and report the changes:

```
ecModel = setProtPoolSize(ecModel);
[ecModel, tunedKcats] = sensitivityTuning(ecModel);
struct2table(tunedKcats)
```

44. Look at the report. The 'tunedKcats' output structure documents which k_{cat} have been modified, together with their previous value and the reaction that is catalyzed by each enzyme. It is worthwhile to inspect this information, to find out whether any of the now-modified k_{cat} values were initially wrongly gathered from any of the k_{cat} sources.

Evaluate the tuned k_{cat} values

▲ **CRITICAL** In Step 43, k_{cat} values are tuned automatically, solely based on their respective enzyme overconstraining the ecModel, preventing it from reaching an intended maximum growth rate. However, there might be good evidence for why the original k_{cat} value was unrealistic and the tuned k_{cat} value in the model is a more likely reflection of the k_{cat} value in

reality. The following steps show a worked example for the k_{cat} of 5′-phosphoribosylformyl glycinamidine synthetase. Repeat these general steps for each k_{cat} value of interest. Note that not every tuned k_{cat} value can be supported by literature data, and alternatively, in vivo apparent enzyme turnover numbers could be considered.

45. On examination, the 'tunedKcats' structure in Step 43 might show that the tuned k_{cat} of 5′-phosphoribosylformyl glycinamidine synthetase (reaction 'r_0079') was much higher than the original value (original 0.05 s^{-1}, tuned 5 s^{-1}). In this example, the original source of the k_{cat} was BRENDA. To gather additional information on how this reaction was matched, get the location of this reaction in the 'kcatList' with:

```
rxnIdx = find(strcmp(kcatList_merged.rxns, 'r_0079'));
kcatList_merged.wildcardLvl(rxnIdx)
kcatList_merged.eccodes(rxnIdx)
kcatList_merged.origin(rxnIdx)
```

This yields a wildcardLvl of 0, EC number 6.3.5.3 and origin 4 (i.e., any organism, any substrate, k_{cat} value).

46. Look at the specific entry in BRENDA. On BRENDA (https://www.brenda-enzymes.org/enzyme.php?ecno=6.3.5.3), the reported k_{cat} of 0.05 is from *E. coli* with NH_3 as substrate. The usual substrate for this enzyme is glutamine, so the reported k_{cat} value might not be a fair estimation for the reaction in the ecModel.
47. Refer to the original paper. In this case, the abstract of the original source[58] states that '(…) NH_3 can replace glutamine as a nitrogen donor with a K_m = 1 M and a turnover of 3 min^{-1} (2% glutamine turnover)'. Meanwhile, the same paper also reports a specific activity with glutamine as substrate, which would be more reasonable to use in the ecModel.
48. Find or calculate the activity of the enzyme with the more suitable substrate. In this example, the specific activity was reported as 2.15 µmol/min/mg protein, and needed to be converted to s^{-1} for use as k_{cat}. As µmol/min/mg protein is the same as mmol/min/g protein, convert to mol/s/g protein as:

```
convKcat = 2.15;
convKcat = convKcat / 1000;
convKcat = convKcat / 60;
```

With the MW of the enzyme, convert to s^{-1}:

```
enzMW = ecModel.ec.mw(strcmp(ecModel.ec.enzymes, 'P38972'));
convKcat = convKcat * enzMW
```

49. Compare the new k_{cat} value with the tuned k_{cat} value. In this case, the converted k_{cat} value is 5.34, which is not too dissimilar to the value of 5 that was reached by 'sensitivityTuning'.
50. Replace the k_{cat} value in the ecModel with the new literature value. Now that an alternative k_{cat} has been calculated for this reaction and enzyme, one could either document it in 'customKcats.tsv' that is used in Step 28, or apply it here directly on the model with:

```
ecModel = setKcatForReactions(ecModel, 'r_0079', 5.34);
ecModel = applyKcatConstraints(ecModel);
```

51. After this process has been repeated for each of the tuned k_{cat} values, consider the results. If convincing evidence exists that the original k_{cat} value is more realistic, it could be considered if, e.g., the stoichiometry of the metabolic network is incomplete as key reactions or even whole pathways might be missing from the starting GEM. Such issues

(similar to Step 38) should be resolved in the conventional GEM before using it as starting GEM for ecModel reconstruction.

52. Save the functional ecModel. The ecModel that is now obtained can be recognized as a functional ecModel. It is able to simulate the physiological data that we have provided. Before progressing with further simulations and analysis with this ecModel (i.e., Stages 4 and 5), it is recommended that the functional ecModel is saved to have it available to load again at any point:

```
saveEcModel(ecModel);
```

Stage 4: integration of proteomics data into the ecModel

● **TIMING 15 min**

Constrain with proteomics data

53. Instead of constraining the total protein content, the concentration of individual enzymes can be constrained by integration of proteomics data. As this process is not suitable for light ecModels, during Stage 4 it is assumed that the ecModel is a full version. It is important to realize that constraining individual enzymes will in most cases generate an overconstrained ecModel, which is not able to reach the intended growth rate, as there are now uncertainties in both the individual enzyme levels and the k_{cat} values. It is therefore imperative that the proteomics data is of high quality, and protein concentrations of lower certainty are discarded.
54. Prepare the proteomics data. It is important to:
    - Provide the protein concentrations in milligram per gram dry cell weight, which can be obtained through various methods[59]
    - Store the data in 'data/proteomics.tsv', which can contain data from multiple experimental conditions
    - Filter out lower certainty data, which include discarding of very low protein concentrations, proteins with high relative standard deviation and proteins absent from most of the replicates
55. Check that the proteomics data is in a standardized file format by referring to 'data/proteomics.tsv'. Similar to integrating k_{cat} in the ecModel, integration of proteomics data involves four steps:
    - Load the proteomics data into a 'protData' structure
    - Populate the 'ecModel.ec.concs' with the relevant enzyme concentrations
    - Apply the enzyme concentrations as constraints in enzyme usage reactions (for reaction direction, see Step 34)
    - Modify the usage reactions of enzymes with defined concentrations to no longer draw from the generic protein pool pseudo-metabolite
    - While it is possible to manually enter and/or modify the 'ecModel.ec.concs' field, we recommend providing the proteomics data in a standardized file format to be read by provided functions.
56. Load the proteomics data set. If the proteomics dataset consists of one experiment with three replicates, load it with:

```
protData = loadProtData(3);
```

If there are two experiments with three replicates each, the command is:

```
protData = loadProtData([3,3]);
```

The above function also includes filtering of the data, which is therefore applied on the whole dataset. When protData contains multiple experiments, specify which experiment should be used (experiment 1 here):

```
ecModel = fillEnzConcs(ecModel, protData, 1);
```

57. In the same way that the ecModel was populated with k_{cat} values, the enzyme concentrations can be filled in using 'ecModel.ec.concs'. Introduce these concentrations to 'ecModel.lb' with:

```
ecModel = constrainEnzConcs(ecModel);
```

▲ **CRITICAL STEP** It is essential that the enzyme concentrations is provided in the correct unit: milligram per gram dry cell weight (DCW). Relative proteomics is not able to provide data in such units. Loading proteomics data with protein levels in incorrect units will result in nonsensical ecModels.

58. Enzymes that are constrained by their concentrations are no longer drawn from the pseudo-metabolite protein pool, which was earlier defined to represent all enzymes combined (Step 32). The constraint on the protein pool exchange reaction should therefore be modified to only represent enzymes that are not constrained by their concentrations. It is also very common that at least some of the listed enzymes will not have available concentrations, due to technical challenges in the experimental methods. To do this, enter the command 'updateProtPool'. As the proteomics data are derived from an experiment where the cells might have changed their physiology, a sample-specific experimentally measured total protein content (e.g., 0.5 g/gDCW) can be explicitly considered, instead of referring to the 'params.Ptot' value in the model adapter, with:

```
ecModel = updateProtPool(ecModel, 0.5);
```

Constrain with experimentally measured exchange fluxes

59. As the integrated enzyme concentrations are derived from a particular experiment, it is relevant to also constrain any exchange reaction for which experimental data are available, e.g., the glucose uptake, ethanol production or CO_2 exchange rate, determined for the same experiment. The flux data file, located at 'data/fluxData.tsv' by default, can contain data from multiple experiments, and can contain as many reaction fluxes as desired. Note that providing a zero flux will result in blocking an exchange reaction. Load the data:

```
fluxData = loadFluxData();
```

60. Decide whether it is most appropriate to use 'loose' constraints or 'strict' constraints for the experimentally measured metabolite exchange rates.
    1. With 'loose' constraints, the measured rate is set as upper bound, while the lower bound is remained at zero (for exchange reactions with a negative rate, this translates to setting the lower bound to the measured value and the upper bound to zero).
    2. With 'strict' constraints, the lower and upper bounds are set at, e.g., 95% and 105% of the measured value.
    The stricter constraints will force the ecModel to adhere to the measured fluxes more closely, but risk that imprecision in, e.g., the measured fluxes or enzyme concentrations result in a nonfunctional model that cannot solve FBA. In that case, the use of 'loose' is advised.
61. Decide how the growth rate should be applied as either 'max' or 'min' constraint.
    - With 'max' the measured growth rate is set as upper bound to indicate the maximum growth rate that the ecModel should be allowed to reach. This is suitable if growth is used as objective function (Step 35)
    - With 'min' the measured growth rate is set as lower bound to indicate the minimum growth rate that the ecModel should be able to reach. This is suitable if the minimization of the protein pool exchange reaction is used as objective function (Step 41)
62. After deciding in Steps 60 and 61 how the experimentally measured fluxes should be constrained in the ecModel (e.g., set 'loose' constraints for the metabolic exchange fluxes and 'max' for the growth rate to yield an ecModel that is least likely to be overconstrained), data from the first experiment are set and FBA is run to predict the growth rate:

```
ecModel = constrainFluxData(ecModel, fluxData, 1, 'max', 'loose');
sol = solveLP(ecModel);
fprintf('Growth rate that is reached: %f /hour\n', abs(sol.f))
```

63. Analyze the results. With 'max' and 'loose' as parameters, the FBA should be able to reach a solution (unless too many zero-exchange fluxes are defined in Step 59). If the resulting ecModel is not able to reach the intended growth rate, it is necessary to make the overconstrained enzyme levels more flexible.

Flexibilize measured protein concentrations

64. Perform an iterative process consisting of calculating the control coefficients of all enzymes and increasing the concentration of the enzyme with the greatest control coefficient. This process is repeated until the intended growth rate (here 0.1 h^{-1}) is reached. It is adjustable by which fold change the enzyme level should be increased in each iteration (here, tenfold). Once the growth rate is reached, reduce the adjusted enzyme levels to the minimum concentration that still allows for the same flux distribution.

```
[ecModel, flexEnz] = flexibilizeEnzConcs(ecModel, 0.1, 10);
```

The flexibilized enzyme levels are reflected in changed constraints of their respective usage_prot reactions (the direction of usage_prot reactions is the same as the protein pool exchange reaction; Step 40).

▲ **CRITICAL STEP** The concentrations in 'ecModel.ec.concs' remain unchanged and continue to reflect the measured values that were obtained via 'fillEnzConcs'. Only the lower bounds of selected usage_prot reactions are changed to the flexibilized value.

65. Inspect which enzymes were modified in 'flexEnz'. While flexibilizing enzyme concentrations is a reasonable approach as it cannot be excluded that the protein measurement was imprecise, it is possible that instead of the enzyme concentration, it should be the k_{cat} that is modified instead. If the same enzyme concentration is repeatedly modified, the user will also be prompted to check its k_{cat} and manually curate it by defining custom k_{cat} values (Step 28 and Step 45, while focusing on the reported problematic enzyme).

Stage 5: simulation and analysis of ecModels

● **TIMING 3 h**

▲ **CRITICAL** As both the light and full ecModels produced by GECKO have the same underlying structure of an *S*-matrix combined with vectors of lower and upper bounds, they can be simulated and analyzed in a same way. Box 2 lists a selection of functions from the RAVEN toolbox[51] that can be applied to both conventional GEMs and ecModels. The ecModels are also suitable for simulations in other constraint-based toolboxes, and SBML files are then the best way to exchange the ecModels (Step 14).

Simulate exchange fluxes at various growth rates

▲ **CRITICAL** The ecModel without proteomics integration can be used to investigate the effect of the total protein pool constraint on metabolism, for instance, at different growth rates. One can postulate that at low growth rates, the cellular metabolic activity is low, and the enzymes are probably present in copious amounts. Contrastingly, at high growth rates metabolic activity is high, such that the high metabolic fluxes maybe become constrained by the amount of available enzyme. The effect of this is shown by simulating exchange fluxes at various growth rates. An example procedure is shown.

66. Load an ecModel without proteomics integration:

```
ecModel = loadEcModel('ecYeastGEM.yml');
```

67. In the 'full_ecModel' tutorial, a custom 'plotCrabtree' function is provided that predicts fluxes at a range of different growth rates and plots selected exchange reactions in a graph,

together with experimentally measured exchange fluxes[60]. Navigate to the 'full_ecModel/code' folder and run with 'plotCrabtree'. The resulting plot is saved in the output subfolder:

```
cd(fullfile(params.path, 'code'))
[fluxes, gRate] = plotCrabtree(ecModel);
saveas(gcf, fullfile(params.path, 'output', 'crabtree.tiff'))
```

▲ **CRITICAL STEP** To be able to run this function, it is essential to first navigate to the 'full_ecModel/code' folder, as this is a custom function that is distributed specific for this ecModel and not part of the general GECKO code.

68. To demonstrate that conventional GEMs are not able to capture this effect, perform these simulations with an ecModel whose protein pool exchange reaction is not constrained. To mimic this, define an infinite cellular protein:

```
ecModel_infProt = setProtPoolSize(ecModel, Inf);
[fluxes, gRate] = plotCrabtree(ecModel_infProt);
saveas(gcf, fullfile(params.path, 'output', 'crabtree_infProt.tiff'))
```

Choose the objective function

▲ **CRITICAL** A common consideration in simulations is the choice of objective function. In FBA of conventional GEMs, specifically when simulating microbes, the maximization of growth is often used as the objective function. This still remains relevant for ecModels but is joined by another critical objective function. Once it is observed what growth rate can be reached, the ecModel should be minimized for the total protein pool reaction, based on the assumption that the cell is more likely to present flux distributions that make efficient use of protein resources.

69. Set maximization of biomass production as the objective function before running FBA:

```
ecModel = setParam(ecModel, 'obj', params.bioRxn, 1);
sol = solveLP(ecModel);
fprintf('Growth rate that is reached: %.4f /hour\n', abs(sol.f))
```

70. Set the growth rate that is reached as lower bound for the biomass production reaction, then change the objective function to minimizing the protein pool exchange reaction (Step 41) and again run FBA. To avoid potential numeric issues due to, e.g., rounding, use 99% of the growth rate as lower bound:

```
ecModel = setParam(ecModel, 'lb', params.bioRxn, 0.99*abs(sol.f));
ecModel = setParam(ecModel, 'obj', 'prot_pool_exchange', 1);
sol = solveLP(ecModel);
fprintf('Minimum protein pool usage: %.2f mg/gDCW\n', abs(sol.f))
```

Such a simulation yields the flux distribution that has a growth rate that is ≥99% of the maximum growth rate in that condition with the most efficient enzyme resource allocation.

Enzyme usage

▲ **CRITICAL** A unique output from full ecModel simulations are enzyme usages: either as absolute usage, corresponding to the flux through the respective usage_prot reactions; or as capacity usage, where the absolute usage is divided by the available protein, as indicated by the constraints of the usage_prot reactions (and not necessarily the values in 'ecModel.ec.concs'; Step 64). Enzyme usages can be particularly insightful when the total protein constraint is limiting, as it details the resource balance allocation across all enzymes. As example, enzyme usage during the Crabtree simulation (Step 67) can be evaluated to see which enzyme contributes most to the total protein constraint.

71. For this, perform a simulation at growth rate of 0.25 h^{-1} with minimizing glucose uptake and subsequentially report the top ten highly used enzymes:

```
ecModel = setParam(ecModel, 'obj', 'r_1714', 1);
ecModel = setParam(ecModel, 'lb', params.bioRxn, 0.25);
sol = solveLP(ecModel, 1);
usageData = enzymeUsage(ecModel, sol.x);
usageReport = reportEnzymeUsage(ecModel, usageData);
usageReport.topAbsUsage
```

Compare flux distributions of different models

▲ **CRITICAL** Flux distributions from ecModels cannot be directly compared with those derived from their starting GEM, as the structure of the model is changed substantially by 'makeEcModel'. It is therefore essential to map the fluxes between the two model formulations.

72. Map back an ecModel derived flux distribution to its starting conventional GEM with 'mapRxnsToConv', where flux values from isozymatic reactions and inverted reversible reactions are combined.

```
sol = solveLP(ecModel);
[mappedFlux, enzUsageFlux, usageEnz] = mapRxnsToConv(ecModel, model, sol.x);
```

The resulting 'mappedFlux' vector has the same number and order of fluxes as what would be obtained from performing FBA on the conventional GEM, and can therefore be treated as any other flux vector derived from a conventional GEM.

▲ **CRITICAL STEP** When mapping reactions from an ecModel back to a conventional GEM, it is essential that the ecModel is indeed derived from the conventional GEM, as the reactions are mapped by their reaction identifiers without further evaluation if the chemical reactions are identical. Any larger stoichiometric changes that are made to ecModel beyond applying enzyme and proteomics constraints will not be correctly reflected. Examples of this are changing the directionality of a reaction, or adding completely new reactions to the ecModel.

Evaluate solution space

▲ **CRITICAL** A major benefit of ecModels is the reduced solution space compared with conventional GEMs. This can be evaluated by FVA, which for each reaction indicates the minimum and maximum rate. As stated above, common analysis methods such as FVA work as well with ecModels as with any other constraint-based metabolic models. In RAVEN toolbox this can be done with 'getAllowedBounds'; however, for comparison with the FVA results from a non-ecModel, it is convenient to directly map the results to the original model, via the 'ecFVA' function.

73. Apply the same exchange flux constraints to three models: (1) an ecModel with proteomics integrated (e.g., obtained from Step 64, indicated here with ecModelProt), (2) the same ecModel without proteomics integrated (e.g., obtained from Step 50, indicated here with ecModel) and (3) their corresponding conventional GEM:

```
ecModelProt = constrainFluxData(ecModelProt, fluxData, 1, 'max', 'loose');
ecModel = constrainFluxData(ecModel, fluxData, 1, 'max', 'loose');
model = constrainFluxData(model, fluxData, 1, 'max', 'loose');
```

The ecFVA results from all three models will be kept in minFlux and maxFlux matrices, which are first defined as zeros before being populated with ecFVA results.

74. Run 'ecFVA':

```
minFlux = zeros(numel(model.rxns),3);
maxFlux = minFlux;
```

```
[minFlux(:,1), maxFlux(:,1)] = ecFVA(model, model);
[minFlux(:,2), maxFlux(:,2)] = ecFVA(ecModel, model);
[minFlux(:,3), maxFlux(:,3)] = ecFVA(ecModelProt, model);
```

75. A range of allowable fluxes can be defined for each reaction by subtracting the minimum flux from the maximum flux. Visualize the resulting flux ranges as cumulative distribution:

```
plotEcFVA(minFlux, maxFlux);
saveas(gca, fullfile(params.path, 'output', 'ecFVA.pdf'))
```

Compare simulations made by full and light ecModel

76. To demonstrate the similarity of simulations by full and light ecModel, a custom function in 'tutorials/full_ecModel/code' generates light and full ecModels for yeast-GEM that are populated with identical k_{cat} values and performs FBA with maximizing growth rate on each ecModel. Map back the flux distributions to the conventional GEM to allow for comparison, which is presented in a scatter plot and the two respective mapped flux vectors:

```
cd(fullfile(findGECKOroot, 'tutorials', 'full_ecModel', 'code'))
[fluxLight, fluxFull] = plotlightVSfull();
```

77. Compare the ratio between the two flux distributions. When inspecting these values, it can be argued that any flux that deviates more than an arbitrary 0.1% between the two ecModels is different:

```
fluxRatio = fluxFull ./ fluxLight;
nonsimilarFlux = abs(fluxRatio - 1) > 0.001;
model.rxnNames(nonsimilarFlux)
```

Context-specific models

▲ **CRITICAL** While ecModels can be reconstructed for higher organisms such as humans, the starting GEM should be not the generic model but rather context-specific models based on omics data by approaches such as tINIT. To obtain context-specific ecModels there are two paths: (i) contextualize the generic model and then convert it to ecModel or (ii) reconstruct a generic ecModel and then contextualize it. The second path has the benefit that any contextualized ecModel derived from the same generic ecModel will have the same k_{cat} data, and the second approach is much faster, which becomes essential when, e.g., reconstructing hundreds of single-cell ecModels. In this step, the demonstration is therefore with the human-GEM in 'tutorials/light_ecModel'. As an example, 'HT29' is a context-specific model of the human colorectal adenocarcinoma cell line HT-29 and 'ecModel' is the generic ecModel, which are both derived from Human-GEM.

78. To reconstruct the context-specific ecModel, run with:

```
ecHT29 = getSubsetEcModel(ecModel, HT29);
```

79. As a simple demonstration that making a model context specific and considering ECs both affect the predicted fluxes, compare the maximum growth rates between the different models:

```
sol = solveLP(HT29);
fprintf('Growth rate in HT29-GEM: %.3f /hour.\n', abs(sol.f))
sol = solveLP(ecModel);
fprintf('Growth rate in ecHuman-GEM: %.3f /hour.\n', abs(sol.f))
sol = solveLP(ecHT29);
fprintf('Growth rate in ecHT29-GEM: %.3f /hour.\n', abs(sol.f))
```

Troubleshooting

Troubleshooting advice can be found in Table 4.

Table 4 | Troubleshooting table

Step	Problem	Possible reason	Solution
9	Various warnings are shown when the conventional GEM is loaded	Not all GEMs in SBML file format strictly adhere to the L3V1 FBCv2 standard. The user can run SBML Validator to get more information on what is wrong with the model	In many cases the model will still load. However, when the model would be exported again to SBML, RAVEN will attempt to avoid writing the same invalid file. It can therefore be useful to run a cycle of 'importModel', 'exportModel' and 'importModel', to ensure that the loaded conventional GEM is as close to valid SBML as possible
12	Complex data is missing for some or all complexes for the species being modeled	Complex Portal does not contain all complexes—in most cases some complexes will be missing	The complex information is not critical for the function of GECKO, the uncertainties in k_{cat} values would be a larger problem than uncertainties in complex stoichiometry. A reasonable option is therefore to skip Step 12 and keep the default stoichiometry of one for all subunits. It is however also possible to manually complement 'ComplexPortal.json' and add complex information collected manually from databases or literature
17	Difficulties to find EC numbers for the reactions	The fuzzy matching requires EC numbers to be matched to reactions	EC number can be entered manually in the 'model.eccodes' field, based on any species-specific information the user might have, and loaded using the 'getECFromGEM' function. In addition, as DLKcat does not require EC numbers, an alternative strategy would be to only use DLKcat as k_{cat} source
24	DLKcat fails to run	Difficulty or not having enough rights to install Docker Desktop	The 'src/dlkcat-gecko/' folder contains all required Python scripts and data to run DLKcat with the 'DLKcat.tsv' output from 'writeDLKcatInput'. Instead of using Docker, all DLKcat requirements can be installed directly in the system terminal (not MATLAB), with: `pipenv install -r requirements.txt` After this, DLKcat can be run from the system terminal via: `pipenv run python DLKcat.py DLKcat.tsv DLKcatOutput.tsv`
28	After running 'applyCustomKcats' there are many entries in 'noMatch'	The k_{cat} values in 'customKcats.tsv' are possibly combined with enzyme annotations, while this does not match to the reactions in the model	Confirm in the ecModel which enzymes are associated to a particular reaction
32	Values to define total protein pool are missing	The protein pool is defined from f, P_{tot}, and sigma	The values can all be assigned to a standard value of 0.5. If the maximum growth rate is too small or large, the sigma parameter can be modified manually until a reasonable maximum growth rate is reached
43	Many k_{cat} values are changed during tuning	Various parameters might not be correctly set	While it can be expected that various k_{cat} values need to be changed (typical is ~20–40), if a large fraction of k_{cat} values are changed, it is more likely that something else is happening
			Ensure that there are no constraints set on any nutrient uptake. Only the protein pool should be limiting
			Confirm that the maximum growth rate as specified in the model adapter as 'obj.params.gR_exp' is realistic
			Test whether the starting GEM is able to simulate the intended growth rate, to check whether the metabolic network stoichiometry is not limiting
54	No access to absolute quantitative proteomics data	Proteomics measurements were not performed with external standards for quantification	Protein quantification through label-free mass spectrometry is a powerful alternative that does not require the use of expensive external standards or stabile isotope labeling[59]. Such data has routinely been used with GECKO analyses
64	The 'flexEnz' structure contains many entries	Lower-quality proteomics data substantially increase the risk of overconstraining the model	Constraining individual enzyme levels substantially reduces the solution space. It is important to confirm that the total protein amount is still realistic and low-quality protein concentrations are filtered out. The filtering the 'loadProtData' can be adjusted via various parameters. By being stricter on which enzyme concentrations are included in the model (e.g., with lower maxRSD, relative s.d.), ensures that only the enzyme levels with the highest certainty are directly constrained, while the other enzyme levels continue to be constrained by the overall protein pool

Timing

The timing of the entire protocols is dependent on the size of the starting GEM and the internet connection.
Stage 0 (Steps 1–7): 15 min
Stage 1 (Steps 8–14): 15 min

Stage 2 (Steps 15–32): 1 h. Deep learning prediction with DLKcat accounts for the most time
Stage 3 (Steps 33–52): 15 min
Stage 4 (Steps 53–65): 15 min. Step 64 accounts for the most time
Stage 5 (Steps 66–79): 3 h. Step 74 accounts for the most time

Anticipated results

The anticipated results described here are the result from running the commands as part of 'tutorials/full_ecModel/protocol.m', if nothing else is indicated. It is important to note that the exact output shown here might be somewhat (numerically) different, dependent on, e.g., updates available at the different databases and various solvers.

Step 1: The adapter folder is initiated at the location that was specified by the user, containing empty subfolders named 'code', 'data', 'models' and 'output', together with a template model adapter file that starts with the specified model name, suffixed with 'Adapter.m'.
Step 2: Typically, the starting GEM file in the 'models' subfolder of the adapter folder.
Steps 3–7: The model adapter file from Step 1 is populated with organism- and model-specific parameters. If required, when no appropriate KEGG and/or UniProt parameters could be identified, custom 'data/uniprotConversion.tsv', 'data/kegg.tsv' and/or 'data/uniprot.tsv' datafiles are constructed in the adapter folder.
Step 8: The MATLAB instance is primed for working with a particular ecModel, by setting the relevant 'modelAdapter' as default.
Step 9: The conventional GEM is loaded into the MATLAB workspace within a structure named 'model'. Simulations with this model can be performed as with any GEM.
Steps 10–11: Enzyme data are downloaded from UniProt and KEGG databases, and stored in 'data/uniprot.tsv' and 'data/kegg.tsv'. In addition, an empty ecModel is generated and named as 'ecModel' in the MATLAB workspace.

A structure called 'noUniprot' may contain a list of genes for which no match could be found in the UniProt database file.

For this worked example, in the 'light_ecModel' tutorial, noUniprot contains six genes that could not be mapped. As this is a relatively low number, it is probably futile to construct different UniProt database files by changing 'obj.params.uniprot.ID' in the model adapter. Instead, manual curation of the gene identifiers, 'uniprotConversion.tsv' or manually adding the relevant entries to the UniProt database file are recommended.

Regardless of noUniprot reporting some genes, an empty ecModel is generated with enzymes for all matching genes. Simulations with this empty ecModel will behave similarly as the conventional GEM it was derived from, as no enzyme and protein pool constraints are applied yet.

Steps 12–13: Enzyme complex data is retrieved and stored in 'data/ComplexPortal.json'. The enzyme complex data is subsequently integrated into the ecModel, modifying the 'ecModel.ec.rxnEnzMat' field with the appropriate stoichiometric coefficients, while a progress bar shows the remaining duration of this process. In the 'full_ecModel' tutorial, the MATLAB command window displays:

```
A total of 206 complex have full match, and 17 proposed.
```

As an example of the content of 'proposedComplex', reaction r_0505_EXP_2 could potentially be matched with complex CPX-1293, named 'mitochondrial 2-oxoglutarate dehydrogenase complex'. In the ecModel, this reaction is annotated with proteins P20967 (KGD1, or subunit E1), P19262 (KGD2, or subunit E2) and P09624 (LPD1, or subunit E3),

while CPX-1293 in Complex Portal also includes the protein P19955 (YMR31), which is known as '37S ribosomal protein'. Indeed, in the literature, evidence can be found that P19955, also termed KGD4 or subunit E4, is an essential subunits that is necessary to form a stable complex[61]. On the basis of this, the starting GEM yeast-GEM should be curated with the updated gene association for reaction r_0505, which would result in a 100% match when running 'makeEcModel' and 'applyComplexData'.

Step 14: The ecModel is stored in the defined file format (e.g., SBML or YAML) in the 'models' subfolder. If stored in SBML, this model file can be loaded by any other relevant software that accepts SBML files.
Steps 15–19: The 'ecModel.ec.eccodes' field is curated to contain EC numbers. k_{cat} data are collected from the BRENDA database and saved as a 'kcatList', e.g., 'kcatList_fuzzy', which contains the following fields:

- source: 'brenda' when the 'kcatList' is generated by 'fuzzyKcatMatching', e.g. 'dlkcat' if generated by 'readDLKcatOutput' (Step 25)
- rxns: reaction identifiers matching 'ecModel.ec.rxns'
- substrates: names of all substrates for each reaction that were used to match to BRENDA database
- kcats: proposed k_{cat} values as derived from fuzzy matching with BRENDA
- eccodes: EC numbers that were used to match to BRENDA database
- wildcardLvl: level of EC wildcard that was necessary to find a match:
  - 0: w.x.y.z
  - 1: w.x.y.-
  - 2: w.x.-.-
  - 3: w.-.-.-
- origin: level of specificity that was necessary to find a match:
  - 1: correct organism, correct substrate, k_{cat} value
  - 2: closest related organism, correct substrate, k_{cat} value
  - 3: correct organism, any substrate, k_{cat} value
  - 4: closest related organism, any substrate, k_{cat} value
  - 5: correct organism, specific activity
  - 6: closest related organism, specific activity

  A progress bar shows the remaining duration of this process.

Steps 20–25: matches that 'findMetSmiles' finds between metabolite names and SMILES are stored in 'data/smilesDB.tsv', to speed up repeated runs of the function on the same ecModel. The 'ecModel.metSmiles' is generated and filled with SMILES. The MATLAB Command Window displays:

```
Check for local SMILES database… done.
SMILES could be found for 64% of the unique metabolite names.
```

An input file for DLKcat is stored in 'data/DLKcat.tsv', containing all required data for k_{cat} prediction. Once 'runDLKcat' is run, predicted k_{cat} values are appended to the same 'DLKcat.tsv' file. As DLKcat runs via a Docker, no progress is reported in the MATLAB Command Window. The resulting 'kcatList' is similar to the one in Step 18, but lacking the here irrelevant 'eccodes', 'wildcardLvl' and 'origin' fields, while instead including a 'genes' field with entries for each k_{cat} prediction.

Step 26: the generated 'kcatList' is similar as those generated in Step 18 and Step 25, with the exception of a 'kcatSource' field that indicates the source for each k_{cat} value individually.
Step 27: the 'ecModel.ec.kcat' field is populated with selected k_{cat} values, with 'ecModel.ec.source' indicating where each k_{cat} is derived from.

Step 28: the 'ecModel.ec.kcat' field is further populated with custom k_{cat} values, overwriting any existing values.
Step 29: the 'ecModel.ec.kcat' field is updated to ensure that all isozymic reactions have k_{cat} values assigned, while 'ecModel.ec.source' for those reactions indicate 'isozymes'. Note that 'ecModel.S' remains unchanged.
Step 30: the 'ecModel.ec' structure is amended to include a 'standard' pseudo-enzyme of MW 'standardMW' (44,898 Da), while reactions indicated in 'rxnsMissingGPR' and 'rxnsNoKcat' have the 'standardKcat' value (12.3 s^{-1}) as their 'ecModel.ec.kcat' entry.
Step 31: the k_{cat} values that are added and/or changed in 'ecModel.ec.kcat' during Steps 27–30 are applied to 'ecModel.S'. The resulting stoichiometric coefficients of the enzyme pseudo-metabolites are defined as $\frac{\text{MW}}{k_{cat} \times 3{,}600} \times (\text{complex stoichiometry})$, where the 3,600 factor converts k_{cat} values from s^{-1} to h^{-1}, to reconcile with the unit of metabolic rates in GEMs and ecModels.

It is important to bear in mind that in 'ecModel.ec', each enzymatic reaction is assigned one k_{cat} value (Table 3), which for reactions catalyzed by enzyme complexes can be the highest predicted k_{cat} across its subunits (Step 27). If assuming the metabolic reaction '$X \rightarrow Y$' is catalyzed by a complex of protein subunits A, B and C at a ratio 1:1:2, all subunits have the same MW of 27,000 Da and the highest predicted k_{cat} value is 30 s^{-1}, then the three subunits are included as pseudo-substrates in the reaction as '$X + 0.25\,A + 0.25\,B + 0.5\,C \rightarrow Y$'.

Step 32: the protein pool exchange reaction is constrained by the calculated maximum enzyme usage. Due to the direction in which the 'prot_pool_exchange' is defined (positive flux would be dissipation from the system, consistent for all exchange reactions), the constraint is implemented as negative lower bound. If the total cellular protein content P_{tot} is 0.5 g/gDCW, the metabolic enzyme fraction over all proteins f is 0.5 and the average saturation σ factor is 0.5, then the 'ecModel.lb' entry for 'prot_pool_exchange' is −125 mg/gDCW.
Steps 33–36: the growth rate immediately after applying the k_{cat} and protein pool constraint is reported in the MATLAB command window, and dependent on the exact k_{cat} values this appears as:

```
Growth rate: 0.107203 /hour
```

As uptake of glucose and other nutrients is not limiting, this value represents the maximum growth rate that the ecModel can reach. Being substantially lower than 0.41 h^{-1} indicates the need for k_{cat} tuning.

Steps 37–38: the maximum growth rate of the starting GEM, without any constraints on carbon uptake, is reported:

```
Growth rate: 19.718033 /hour
```

While this growth rate is not biologically realistic (due to the unconstrained carbon uptake), it does signify that the stoichiometry of the conventional GEM is able to reach the maximum growth rate of 0.41 h^{-1}, indicating that the problem in Step 35 is due to the ECs.

Steps 39–41: the 'ecModel.lb' entry for 'prot_pool_exchange' is set to −1,000 mg/gDCW, which implies that the whole cell is 100% protein. While unrealistic, this often avoids the ecModel being overconstrained and not reaching the intended maximum growth rate. In this simulation is shown:

```
Protein pool usage is: 446 mg/gDCW
```

Taking into consideration the f factor of 0.5, the protein pool usage here implies that the cell would consist of 89.2% protein. That is, assuming a full saturation of all enzymes, if the σ factor would be lower (which is more realistic), then the simulated total protein content would

Table 5 | Example output from 'sensitivityTuning' in the full_ecModel tutorial

rxns	rxnNames	enzymes	oldKcat	newKcat	source
r_0079	5′-Phosphoribosylformyl glycinamidine synthetase	P38972	0.05	5	BRENDA
r_0109	Acetyl-CoA carboxylase, reaction	P48445; Q00955	1.23	12.3	BRENDA
r_0450	Fructose-bisphosphate aldolase	P14540	32.5	325	Custom
r_0486_EXP_2	Glyceraldehyde-3-phosphate dehydrogenase	P00360	29	290	Custom
r_0698	Lanosterol synthase	P38604	0.001937	0.1937	BRENDA
r_1166_EXP_11	Glucose transport	P38695	19.26	192.6	DLKcat

be more than 100%, exemplifying why this approach of simply releasing the protein pool constraint is not recommended.

Steps 43–44: the MATLAB command window shows how iterative tuning of selected k_{cat} values increases the growth rate until it approaches the intended value of 0.41 h^{-1}:

```
Iteration 1: Growth: 0.11421
Iteration 2: Growth: 0.18016
Iteration 3: Growth: 0.23412
Iteration 4: Growth: 0.27495
Iteration 5: Growth: 0.30635
Iteration 6: Growth: 0.33961
Iteration 7: Growth: 0.3718
Iteration 8: Growth: 0.39141
Iteration 9: Growth: 0.41193
```

Selected 'ecModel.ec.kcat' entries, as indicated in 'tunedKcats' (Table 5), are adjusted to yield an ecModel that reaches the intended maximum growth rate, and the new k_{cat} values are applied to 'ecModel.S'. The corresponding 'ecModel.ec.source' still refers to the original source of the k_{cat} value, so it is recommended to inspect the 'tunedKcats' output and already include the required k_{cat} adjustments in Step 28. An example of confirming the new k_{cat} values is shown in Steps 45–52.

Steps 45–52: additional information on the k_{cat} source for reaction r_0079 (Table 5) as gathered from the 'kcatList' structure indicates that the wildcardLvl was 0, the origin was 4 (see 'Anticipated results' Steps 15–19). Querying BRENDA for this reaction and conversion of the reported specific activity to k_{cat} yields a new value of 5.34 s^{-1}, matching the value found by 'sensitivityTuning' (Table 5). The new k_{cat} value is applied to the ecModel directly, with 'setKcatForReactions', but best practice is to include those curated k_{cat} values in 'customKcat.tsv' (Step 28).
Steps 53–57: a 'protData' structure contains the proteomics data as provided in 'data/proteomics.tsv', or could be manually reconstructed from other sources. The structure exists of two fields, abundances contains proteomics data where columns are different datasets while rows refer to proteins, which are identified in the 'uniprotIDs' field. The 'ecModel.ec.concs' field is populated by 'fillEnzConcs'. After 'constrainEnzConcs', the enzymes with concentrations do no longer draw from the 'prot_pool', but are rather constrained by setting the 'ecModel.lb' of the relevant reactions to minus their concentration (again due to the direction of those reactions: negative flux implies enzyme usage by the ecModel).
Step 58: a 'fluxData' structure contains the reaction flux and total protein data as provided in 'data/fluxes.tsv', or could be manually reconstructed from other sources. The structure contains of the fields:

- conds: string of text to name a particular condition with flux data
- Ptot: total protein content in g/gDCW
- grRate: growth rate in h^{-1}
- exchFluxes: numeric values of exchange reaction rates in mmol/gDCWh

- exchMets: names of the metabolites matching the exchFluxes exchange reactions
- exchRxnIDs: reaction identifiers matching the exchFluxes exchange reactions
  The recalculated 'prot_pool' constraint (first set in Step 32) is applied in 'ecModel.lb' by 'updateProtPool'.

Steps 59–63: the relevant entries in 'ecModel.lb' and 'ecModel.ub' are updated, dependent on the content of 'fluxData' and the settings chose when running 'constrainFluxData'. The MATLAB command window reports:

```
Growth rate that is reached: 0.004578 /hour
```

Being far less than the intended growth rate of 0.1 h^{-1} that is specified in 'fluxData.grRate'.

Steps 64–65: the 'flexibilizeEnzConcs' function reports on its progress in the MATLAB Command Window, detailing the enzyme whose concentration is flexibilized and the new growth rate that can now be achieved. When reaching the intended growth rate, or the maximum iterations per enzyme ('iterPerEnzyme', default 5), the 'ecModel.ec.concs' and 'ecModel.S' structures are updated. The updated concentrations, also reported in 'flexEnz', are those that were required in the final growth simulation performed by 'flexibilizeEnzConcs'.

In the 'full_ecModel' tutorial, flexibilization of the enzyme concentrations is not sufficient to reach the intended growth rate of 0.1 $hour^{-1}$ (abridged output shown here):

```
Protein P37299 LB adjusted. Grow: 0.0097884
Protein Q03028 LB adjusted. Grow: 0.012172
Protein P24521 LB adjusted. Grow: 0.012897
Protein P00128 LB adjusted. Grow: 0.013333
[…]
Protein P32473 LB adjusted. Grow: 0.088025
Protein P07285 LB adjusted. Grow: 0.088895
Protein P12695 LB adjusted. Grow: 0.088899
Protein P36148 LB adjusted. Grow: 0.0889
No (more) limiting enzymes have been found. Attempt to increase protein
pool exchange…
Protein pool exchange was also not limiting. Inability to reach growth
rate is not related to enzyme constraints. Maximum growth rate is 0.0889.
```

When no individual enzyme concentrations are preventing the intended growth rate to be reached, the protein pool exchange constraint is released (i.e. lower bound set to −1,000, see Step 34). However, the inability to reach a growth rate of 0.1 $hour^{-1}$ seems not related to enzyme constraints. Indeed simulating the same condition in the conventional GEM:

```
model = constrainFluxData(model,fluxData);
sol = solveLP(model);
fprintf('Growth rate that is reached: %f /hour\n', abs(sol.f))
```

Gives as output:

```
Growth rate that is reached: 0.0889 /hour
```

This confirms that indeed the growth rate is over-constrained by exchange rates, possibly due to measurement errors, rather than enzyme constraints.

Steps 66–68: simulating the Crabtree effect in various ecModels demonstrates the effect of the total protein constraint (Fig. 3). Besides running 'plotCrabtree' on the ecModel

with infinite protein pool (mimicking a conventional GEM, Fig. 3a), which demonstrates that such models are unable to predict the shift to fermentation at high growth rate, the same simulation can be applied on the ecModel before sensitivity tuning of the k_{cat} values in Step 43 (Fig. 3b). This demonstrates that if some selected k_{cat} values are too low, the ecModel is unable to simulate higher growth rates. Note that only 13 k_{cat} values had to be tuned (Table 5, separating Fig. 3a from 3b) for realistic maximum growth rates to be reached.

Then, the ecModel with tuned k_{cat} values exhibits the Crabtree effect at high growth rates (Fig. 3c): yeast switches from respiration (represented by oxygen exchange) to fermentation (represented by ethanol exchange). This switch occurs when the protein pool becomes fully used (Fig. 3d) and thereby constraining the energetically more efficient respiratory flux distribution.

While the ecModel simulation shows the same overall trend as the experimental data, the protein pool seems to become limiting at a growth rate that is too low, e.g., the slope of the simulated glucose uptake rate increases after growth rate 0.2 h^{-1}, while this should happen from around 0.28 h^{-1} according to the experimental data. It is possible that one enzyme is dominating the protein pool constraint too early. Inspecting the top used enzymes may be informative, as done in Step 71.

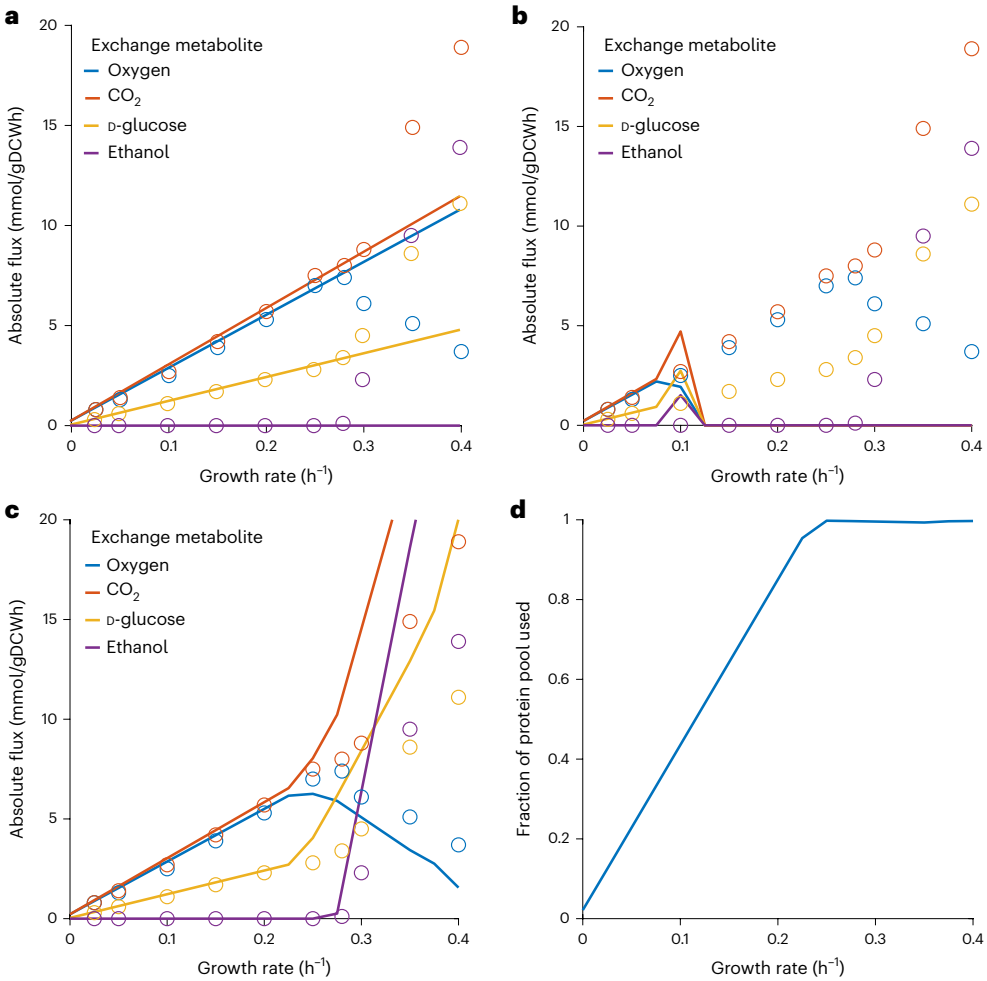

Fig. 3 | Simulations of Crabtree effect in various models. Absolute fluxes through selected exchange reactions, both from simulations (lines) and experimentally measured fluxes (circles). **a**, ecModel without protein pool constraint, mimicking a conventional GEM. **b**, ecModel before k_{cat} sensitivity tuning. **c**, ecModel after k_{cat} sensitivity tuning. **d**, the fraction of protein pool in the ecModel that is used, complementing simulation shown in panel C.

Table 6 | Top ten enzyme usages at growth rate 0.25 h^{-1}

protID	absUse	percUse	k_{cat}	Source	rxnID	rxnNames
P23641	8.7	7.0	12.3	Standard	r_1245_EXP_2	Phosphate transport
P0CD90	5.0	4.0	12.3	Standard	r_1277_EXP_1	Water diffusion
Q12233	4.7	3.8	120	Custom	r_0226	ATP synthase
P36148	4.4	3.5	0.0918	BRENDA	r_2308_EXP_2	Glycerol-3-phosphate acyltransferase (16:0), ER membrane
P05694	3.6	2.9	0.33	BRENDA	r_0085	5-Methyltetrahydropteroyltriglutamate-homocysteine *S*-methyltransferase
P07256	3.3	2.7	50.8	DLKcat	r_0439_EXP_1	Ubiquinol:ferricytochrome c reductase
P32895	3.3	2.6	0.66	BRENDA	r_0916_EXP_1	Phosphoribosylpyrophosphate synthetase
P00163	2.9	2.3	50.8	DLKcat	r_0439_EXP_1	Ubiquinol:ferricytochrome c reductase
P33312	2.8	2.3	0.000667	BRENDA	r_0015	2,5-Diamino-6-ribosylamino-4(3H)-pyrimidinone 5-phosphate reductase (NADPH)
P07257	2.7	2.1	50.8	DLKcat	r_0439_EXP_1	Ubiquinol:ferricytochrome c reductase

protID is the UniProt protein identifier, absUse is the absolute enzyme usage (in mg/gDCW), percUse is the percentage of the total protein pool being allocated to this enzyme. To reduce table size, not all columns of 'usageReport.topAbsUsage' are shown, and only one entry per enzyme is shown (as multiple reactions can be catalyzed by the same enzyme, the output usually shows multiple rows per enzyme, one for each reaction).

Steps 69–70: the ecModel is constraint to perform at least 99% of the maximum growth rate, while FBA minimizes the protein pool usage.

```
Growth rate that is reached: 0.4121 /hour
Minimum protein pool usage: 123.75 mg/gDCW
```

Step 71: the ecModel is constrained to perform at least a growth rate of 0.25 h^{-1}, while minimizing the glucose exchange. The 'usageData' structure contains the absolute and capacity enzymes usages from the FBA simulation, while the 'usageReport' structure organizes this data in 'topAbsUsage' and 'highCapUsage'. As the ecModel used in this simulation has concentrations in 'ecModel.ec.concs', the 'highCapUsage' structure is empty. The 'topAbsUsage' shows the top ten enzymes usages (Table 6).
Step 72: the flux distribution from ecModel is mapped back to the conventional GEM, so that 'mappedFlux' is a vector with the same length (4,063 entries) as 'model.rxns', where the flux values correspond to the order in 'model'. The 'enzUsageFlux' vector contains the enzyme usage fluxes that are not considered in conventional GEMs and are therefore separately reported, while the 'usageEnz' vector contains the protein identifiers that correspond to the 'enzUsageFlux' vector.
Steps 73–75: the 'minFlux' and 'maxFlux' matrices have three columns each, corresponding to the three models, and refer to the reactions as they are ordered in 'model.rxns'. Visualization of the flux variability (Fig. 4), stored in 'output/ecFVA.pdf', shows that the median flux variability

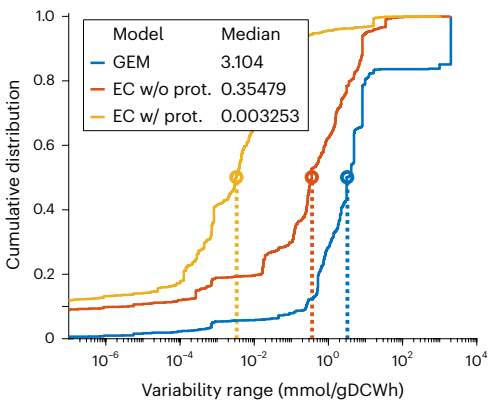

Fig. 4 | Flux variability of various models. Cumulative distributions for three models: a conventional GEM, its derived ecModel without proteomics (w/o prot.) integrated and the same ecModel now constrained by proteomics. The variability range is defined as the maximum flux minus the minimum flux through a particular reaction. Median variabilities are indicated for each model by the dotted line.

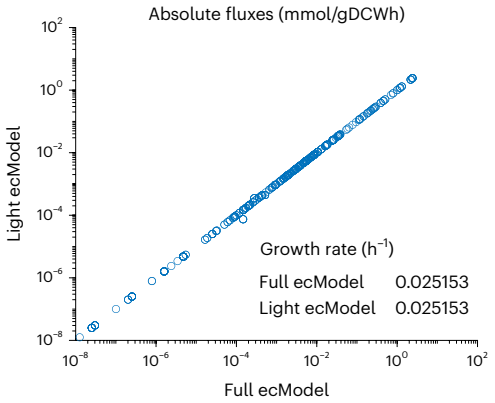

Fig. 5 | Comparison of fluxes in full and light ecModels. The two models reach the same growth rate. Each circle represents the flux through a particular reaction after mapping to the conventional GEM. Only minor deviations are observed, which are reactions involved in lipid metabolism.

of the 'ecModel' is more than four-fold lower than in the conventional GEM. Moreover, the conventional GEM has more than 15% of reactions with very high flux variability (~1,000 mmol/gDCWh), while this high variability is absent from the ecModels.

Steps 76–77: a light and full ecModel of yeast-GEM are made, populated only with k_{cat} values from 'fuzzyKcatMatching', to minimize additional changes between the two ecModel formats. The progress and duration of the various steps is reported in the MATLAB command window:

```
Comparison of duration light vs. full ecModel
ecModel reconstruction: 95% (39 vs 41 seconds)
FBA: 64% (0.57 vs 0.89 seconds)
Mapping fluxes: 64% (0.103 vs 0.161 seconds)
Growth rate that is reached: 0.0252 vs 0.0252
```

While ecModel reconstruction does not drastically differ between the two ecModel styles, as both require the time-demanding 'fuzzyKcatMatching', FBA and subsequent mapping of the fluxes back to the conventional GEM reactions are substantially faster in light ecModels. The light ecModel is therefore particularly useful to cases where many simulations would be performed, e.g., extensive ecFVA or simulation of numerous single-cell ecModels. In other cases, if proteomics data might be integrated with the ecModel, it can be justified to make a full ecModel as it will not drastically affect performance but enable constraining individual enzymes.

The generated flux distributions are near identical for the two ecModels (Fig. 5). The only reactions whose fluxes deviated more than 0.1% between the two ecModels were involved in lipid metabolism, of which a selection is shown here:

```
PE methyltransferase (1-16:0, 2-16:1), ER membrane
PE methyltransferase (1-16:1, 2-16:1), ER membrane
PE methyltransferase (1-18:0, 2-16:1), ER membrane
PE methyltransferase (1-16:0, 2-18:1), ER membrane
PE methyltransferase (1-16:1, 2-18:1), ER membrane
PE methyltransferase (1-18:0, 2-18:1), ER membrane
[...]
lysoPE (1-16:0) oleoyl-CoA acyltransferase, lipid particle
lysoPE (1-16:1) oleoyl-CoA acyltransferase, lipid particle
lysoPE (1-18:0) oleoyl-CoA acyltransferase, lipid particle
PE phospholipase A2 (1-16:0, 2-16:1), lipid particle
```

```
PE phospholipase A2 (1-16:1, 2-16:1), lipid particle
PE phospholipase A2 (1-18:0, 2-16:1), lipid particle
[…]
```

Lipid metabolism in yeast-GEM are defined following the SLIMEr formulation[62], where reactions for each potential fatty acyl-chain configuration are explicitly included, as seen in the output above. The resulting alternative reactions are all catalyzed by the same enzyme and therefore require identical enzyme usages. This introduces a flexibility in the model where there are multiple paths through lipid metabolism that all reach the same overall acyl-chain distribution. This flexibility that is inherited from the conventional GEM is not affected by the ECs, once more demonstrating that ecModels can be used as realistic drop-in alternatives for analyses of conventional GEMs.

Steps 78–79: the resulting 'ecHT29' is based on the generic ecModel and therefore contains all its enzyme and k_{cat} specific information, for those reactions and genes that were present in the contextualized generic GEM HT29. A comparison of the maximum growth rates shows the impact of both ECs and contextualization:

```
Growth rate in HT29-GEM: 149.97 /hour.
Growth rate in ecHuman-GEM: 0.122 /hour.
Growth rate in ecHT29-GEM: 0.086 /hour.
```

Data availability

All (limited) data used in the tutorials are distributed as part of the code (see below).

Code availability

The source code of the GECKO toolbox is publicly available under the MIT license at: https://github.com/SysBioChalmers/GECKO/, also available at https://doi.org/10.5281/zenodo.7699818.

Received: 5 March 2023; Accepted: 13 October 2023;
Published online: 18 January 2024

References

1. Kim, W. J., Kim, H. U. & Lee, S. Y. Current state and applications of microbial genome-scale metabolic models. *Curr. Opin. Syst. Biol.* **2**, 10–18 (2017).
2. Chen, Y., Li, G. & Nielsen, J. Genome-scale metabolic modeling from yeast to human cell models of complex diseases: latest advances and challenges. *Methods Mol. Biol.* **2049**, 329–345 (2019).
3. Clark, T. J., Guo, L., Morgan, J. & Schwender, J. Modeling plant metabolism: from network reconstruction to mechanistic models. *Annu. Rev. Plant Biol.* **71**, 303–326 (2020).
4. Fang, X., Lloyd, C. J. & Palsson, B. O. Reconstructing organisms in silico: genome-scale models and their emerging applications. *Nat. Rev. Microbiol.* https://doi.org/10.1038/s41579-020-00440-4 (2020).
5. Heinken, A., Basile, A., Hertel, J., Thinnes, C. & Thiele, I. Genome-scale metabolic modeling of the human microbiome in the era of personalized medicine. *Annu Rev. Microbiol* **75**, 199–222 (2021).
6. Lewis, N. E., Nagarajan, H. & Palsson, B. O. Constraining the metabolic genotype–phenotype relationship using a phylogeny of in silico methods. *Nat. Rev. Microbiol.* **10**, 291–305 (2012).
7. Lloyd, C. J. et al. COBRAme: a computational framework for genome-scale models of metabolism and gene expression. *PLoS Comput. Biol.* **14**, e1006302 (2018).
8. Goelzer, A. et al. Quantitative prediction of genome-wide resource allocation in bacteria. *Metab. Eng.* **32**, 232–243 (2015).
9. Salvy, P. & Hatzimanikatis, V. The ETFL formulation allows multi-omics integration in thermodynamics-compliant metabolism and expression models. *Nat. Commun.* **11**, 30 (2020).
10. Chen, Y. et al. Proteome constraints reveal targets for improving microbial fitness in nutrient-rich environments. *Mol. Syst. Biol.* **17**, e10093 (2021).
11. Beg, Q. K. et al. Intracellular crowding defines the mode and sequence of substrate uptake by *Escherichia coli* and constrains its metabolic activity. *Proc. Natl Acad. Sci. USA* **104**, 12663–12668 (2007).
12. Adadi, R., Volkmer, B., Milo, R., Heinemann, M. & Shlomi, T. Prediction of microbial growth rate versus biomass yield by a metabolic network with kinetic parameters. *PLoS Comput. Biol.* **8**, e1002575 (2012).
13. Sánchez, B. J. et al. Improving the phenotype predictions of a yeast genome-scale metabolic model by incorporating enzymatic constraints. *Mol. Syst. Biol.* **13**, 935 (2017).
14. Bekiaris, P. S. & Klamt, S. Automatic construction of metabolic models with enzyme constraints. *BMC Bioinforma.* **21**, 19 (2020).
15. Mao, Z. et al. ECMpy, a simplified workflow for constructing enzymatic constrained metabolic network model. *Biomolecules* **12**, 65 (2022).
16. Domenzain, I. et al. Reconstruction of a catalogue of genome-scale metabolic models with enzymatic constraints using GECKO 2.0. *Nat. Commun.* **13**, 1–13 (2022).
17. Chen, Y. & Nielsen, J. Mathematical modelling of proteome constraints within metabolism. *Curr. Opin. Syst. Biol.* **25**, 50–56 (2021).
18. Kerkhoven, E. J. Advances in constraint-based models: methods for improved predictive power based on resource allocation constraints. *Curr. Opin. Microbiol.* **68**, 102168 (2022).
19. Chen, Y. & Nielsen, J. Energy metabolism controls phenotypes by protein efficiency and allocation. *Proc. Natl Acad. Sci. USA* **116**, 17592–17597 (2019).
20. Gustafsson, J. et al. Metabolic collaboration between cells in the tumor microenvironment has a negligible effect on tumor growth. Preprint at *bioRxiv* https://doi.org/10.1101/2022.02.08.479584 (2022).

21. Chen, Y. & Nielsen, J. Yeast has evolved to minimize protein resource cost for synthesizing amino acids. *Proc. Natl Acad. Sci. USA* **119**, e2114622119 (2022).
22. Chang, A. et al. BRENDA, the ELIXIR core data resource in 2021: new developments and updates. *Nucleic Acids Res.* **49**, D498–D508 (2021).
23. Davidi, D. & Milo, R. Lessons on enzyme kinetics from quantitative proteomics. *Curr. Opin. Biotechnol.* **46**, 81–89 (2017).
24. Li, F. et al. Deep learning-based kcat prediction enables improved enzyme-constrained model reconstruction. *Nat. Catal.* https://doi.org/10.1038/s41929-022-00798-z (2022).
25. Sulheim, S. et al. Enzyme-constrained models and omics analysis of *Streptomyces coelicolor* reveal metabolic changes that enhance heterologous production. *iScience* **23**, 101525 (2020).
26. Zhou, J., Zhuang, Y. & Xia, J. Integration of enzyme constraints in a genome-scale metabolic model of *Aspergillus niger* improves phenotype predictions. *Microb. Cell Fact.* **20**, 125 (2021).
27. Ye, C. et al. Improving lysine production through construction of an *Escherichia coli* enzyme-constrained model. *Biotechnol. Bioeng.* **117**, 3533–3544 (2020).
28. Chen, Y. et al. Genome-scale modeling for *Bacillus coagulans* to understand the metabolic characteristics. *Biotechnol. Bioeng.* **117**, 3545–3558 (2020).
29. Lu, H. et al. Yeast metabolic innovations emerged via expanded metabolic network and gene positive selection. *Mol. Syst. Biol.* **17**, e10427 (2021).
30. Ishchuk, O. P. et al. Genome-scale modeling drives 70-fold improvement of intracellular heme production in *Saccharomyces cerevisiae*. *Proc. Natl Acad. Sci. USA* **119**, e2108245119 (2022).
31. Li, Z. et al. Systems engineering of *Escherichia coli* for high-level shikimate production. *Metab. Eng.* **75**, 1–11 (2023).
32. Bujdoš, D. et al. Engineering of *Pseudomonas putida* for accelerated co-utilization of glucose and cellobiose yields aerobic overproduction of pyruvate explained by an upgraded metabolic model. *Metab. Eng.* **75**, 29–46 (2023).
33. Domenzain, I., Lu, Y., Shi, J., Lu, H. & Nielsen, J. Computational biology predicts metabolic engineering targets for increased production of 102 valuable chemicals in yeast. Preprint at *bioRxiv* https://doi.org/10.1101/2023.01.31.526512 (2023).
34. Heirendt, L. et al. Creation and analysis of biochemical constraint-based models using the COBRA Toolbox v.3.0. *Nat. Protoc.* https://doi.org/10.1038/s41596-018-0098-2 (2019).
35. Orth, J. D., Thiele, I. & Palsson, B. O. What is flux balance analysis? *Nat. Biotechnol.* **28**, 245 (2010).
36. Gudmundsson, S. & Thiele, I. Computationally efficient flux variability analysis. *BMC Bioinforma.* **11**, 489 (2010).
37. Choi, H. S., Lee, S. Y., Kim, T. Y. & Woo, H. M. In silico identification of gene amplification targets for improvement of lycopene production. *Appl. Environ. Microbiol.* **76**, 3097–3105 (2010).
38. Agren, R. et al. Identification of anticancer drugs for hepatocellular carcinoma through personalized genome-scale metabolic modeling. *Mol. Syst. Biol.* **10**, 721 (2014).
39. Cho, J. S., Gu, C., Han, T. H., Ryu, J. Y. & Lee, S. Y. Reconstruction of context-specific genome-scale metabolic models using multiomics data to study metabolic rewiring. *Curr. Opin. Syst. Biol.* **15**, 1–11 (2019).
40. Robinson, J. L. et al. An atlas of human metabolism. *Sci. Signal.* **13**, 1482 (2020).
41. Yeo, H. C., Hong, J., Lakshmanan, M. & Lee, D. Y. Enzyme capacity-based genome scale modelling of CHO cells. *Metab. Eng.* **60**, 138–147 (2020).
42. Wendering, P. & Nikoloski, Z. Genome-scale modeling specifies the metabolic capabilities of *Rhizophagus irregularis*. *mSystems* **7**, e0121621 (2022).
43. Wu, K. et al. ecBSU1: A genome-scale enzyme-constrained model of *Bacillus subtilis* based on the ECMpy workflow. *Microorganisms* **11**, 178 (2023).
44. Österberg, L. et al. A novel yeast hybrid modeling framework integrating Boolean and enzyme-constrained networks enables exploration of the interplay between signaling and metabolism. *PLoS Comput. Biol.* **17**, e1008891 (2021).
45. Chen, Y., Li, F. & Nielsen, J. Genome-scale modeling of yeast metabolism: retrospectives and perspectives. *FEMS Yeast Res* **22**, 1–9 (2022).
46. The UniProt Consortium. UniProt: the universal protein knowledgebase in 2023. *Nucleic Acids Res.* **51**, D523–D531 (2023).
47. Kanehisa, M., Furumichi, M., Sato, Y., Ishiguro-Watanabe, M. & Tanabe, M. KEGG: integrating viruses and cellular organisms. *Nucleic Acids Res* **49**, D545–D551 (2021).
48. Thiele, I. & Palsson, B. Ø. A protocol for generating a high-quality genome-scale metabolic reconstruction. *Nat. Protoc.* **5**, 93–121 (2010).
49. Norsigian, C. J., Fang, X., Seif, Y., Monk, J. M. & Palsson, B. O. A workflow for generating multi-strain genome-scale metabolic models of prokaryotes. *Nat. Protoc.* https://doi.org/10.1038/s41596-019-0254-3 (2019).
50. Machado, D., Andrejev, S., Tramontano, M. & Patil, K. R. Fast automated reconstruction of genome-scale metabolic models for microbial species and communities. *Nucleic Acids Res* **46**, 7542–7553 (2018).
51. Wang, H. et al. RAVEN 2.0: a versatile toolbox for metabolic network reconstruction and a case study on *Streptomyces coelicolor*. *PLoS Comput. Biol.* **14**, e1006541 (2018).
52. Davidi, D. et al. Global characterization of in vivo enzyme catalytic rates and their correspondence to in vitro kcat measurements. *Proc. Natl Acad. Sci. USA* **113**, 3401–3406 (2016).
53. Küken, A., Gennermann, K. & Nikoloski, Z. Characterization of maximal enzyme catalytic rates in central metabolism of *Arabidopsis thaliana*. *Plant J.* **103**, 2168–2177 (2020).
54. Heckmann, D. et al. Kinetic profiling of metabolic specialists demonstrates stability and consistency of in vivo enzyme turnover numbers. *Proc. Natl Acad. Sci. USA* **117**, 23182–23190 (2020).
55. Chen, Y. & Nielsen, J. In vitro turnover numbers do not reflect in vivo activities of yeast enzymes. *Proc. Natl Acad. Sci. USA* **118**, 2021 (2021).
56. Xu, R., Razaghi-Moghadam, Z. & Nikoloski, Z. Maximization of non-idle enzymes improves the coverage of the estimated maximal in vivo enzyme catalytic rates in *Escherichia coli*. *Bioinformatics* https://doi.org/10.1093/BIOINFORMATICS/BTAB575 (2021).
57. Heckmann, D. et al. Machine learning applied to enzyme turnover numbers reveals protein structural correlates and improves metabolic models. *Nat. Commun.* **9**, 5252 (2018).
58. Schendel, F., Mueller, E., Stubbe, J., Shiau, A. & Smith, J. Formylglycinamide ribonucleotide synthetase from *Escherichia coli*: cloning, sequencing, overproduction, isolation, and characterization. *Biochemistry* **28**, 2459–2471 (1989).
59. Sánchez, B. J. et al. Benchmarking accuracy and precision of intensity-based absolute quantification of protein abundances in *Saccharomyces cerevisiae*. *Proteomics* **21**, e2000093 (2021).
60. Van Hoek, P., Van Dijken, J. P. & Pronk, J. T. Effect of specific growth rate on fermentative capacity of baker's yeast. *Appl. Environ. Microbiol.* **64**, 4226–4233 (1998).
61. Heublein, M. et al. The novel component Kgd4 recruits the E3 subunit to the mitochondrial α-ketoglutarate dehydrogenase. *Mol. Biol. Cell* **25**, 3342–3349 (2014).
62. Sánchez, B. J., Li, F., Kerkhoven, E. J. & Nielsen, J. SLIMEr: probing flexibility of lipid metabolism in yeast with an improved constraint-based modeling framework. *BMC Syst. Biol.* **13**, 4 (2019).
63. Lewis, N. E. et al. Omic data from evolved *E. coli* are consistent with computed optimal growth from genome-scale models. *Mol. Syst. Biol.* **6**, 390 (2010).

Acknowledgements
The Novo Nordisk Foundation (grant no. NNF20CC0035580); the Knut and Alice Wallenberg Foundation; the European Union's Horizon 2020 research and innovation program (grant agreements 720824 and 814650); the Research Council for Environment, Agricultural Sciences, and Spatial Planning (Formas, grant 2018-00597); and the Swedish Research Council (VR, grant 2019-04624).

Author contributions
J.G., A.T.R., M.A. and E.J.K. wrote the code. Y.C., J.G., A.T.R., M.A., I.D., C.K., F.L., L.Y. and E.J.K. contributed to the software design. Y.C., C.K. and F.L. tested the protocol. Y.C., J.G. and E.J.K. drafted the manuscript. A.T.R., M.A., I.D., C.K., F.L., L.Y. and J.N. edited the manuscript. E.J.K. supervise the project.

Competing interests
The authors declare no competing interests.

Additional information
Correspondence and requests for materials should be addressed to Eduard J. Kerkhoven.

Peer review information *Nature Protocols* thanks Sriram Chandrasekaran and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Related links
Key references using this protocol
Domenzain, I. et al. *Nat. Commun.* **13**, 3766 (2022): https://doi.org/10.1038/s41467-022-31421-1
Li, F. et al. *Nat. Catal.* **5**, 662–672 (2022): https://doi.org/10.1038/s41929-022-00798-z
Sánchez, B. J. et al. *Mol. Syst. Biol.* **13**, 935 (2017): https://doi.org/10.15252/msb.20167411

© Springer Nature Limited 2024